



UNIVERSIDAD SAN FRANCISCO DE QUITO USFQ

Colegio de Ciencias e Ingenierías

Sistema Adversarial de Simulación de Ataques con Clasificación

Automática de Tácticas MITRE ATT&CK Mediante Agentes Autónomos

Francisco Jesús Alarcón Aguirre

Ingeniería en Ciencias de la Computación

Trabajo de fin de carrera presentado como requisito
para la obtención del título de Ingeniero en Ciencias de la Computación

Tutor: Roberto Andrade

Quito, abril de 2026

© DERECHOS DE AUTOR

Por medio del presente documento certifico que he leído todas las Políticas y Manuales de la Universidad San Francisco de Quito USFQ, incluyendo la Política de Propiedad Intelectual USFQ, y estoy de acuerdo con su contenido, por lo que los derechos de propiedad intelectual del presente trabajo quedan sujetos a lo dispuesto en esas Políticas.

Asimismo, autorizo a la USFQ para que realice la digitalización y publicación de este trabajo en el repositorio virtual, de conformidad a lo dispuesto en la Ley Orgánica de Educación Superior del Ecuador.

Nombres y apellidos: Francisco Jesús Alarcón Aguirre

Código: 00324826

Cédula de identidad: 1727730630

Lugar y fecha:

Quito, 28 de febrero del 2026

ACLARACIÓN PARA PUBLICACIÓN

Nota: El presente trabajo, en su totalidad o cualquiera de sus partes, no debe ser considerado como una publicación, incluso a pesar de estar disponible sin restricciones a través de un repositorio institucional. Esta declaración se alinea con las prácticas y recomendaciones presentadas por el Committee on Publication Ethics COPE descritas por Barbour et al. (2017) Discussion document on best practice for issues around theses publishing, disponible en <http://bit.ly/COPETheses>.

UNPUBLISHED DOCUMENT

Note: The following capstone project is available through Universidad San Francisco de Quito USFQ institutional repository. Nonetheless, this project – in whole or in part – should not be considered a publication. This statement follows the recommendations presented by the Committee on Publication Ethics COPE described by Barbour et al. (2017) Discussion document on best practice for issues around theses publishing available on <http://bit.ly/COPETheses>.

RESUMEN

La creciente aplicación de Large Language Models (LLMs) a tareas de razonamiento complejo ha abierto nuevas oportunidades en ciberseguridad, tanto en automatización ofensiva como en monitoreo defensivo. Este trabajo NO propone una nueva arquitectura agéntica como contribución per se, sino que mide empíricamente la precisión de inferencia táctica MITRE ATT&CK de un observer LLM mediante telemetría de red y logs del sistema en un entorno adversarial controlado, dentro del régimen documentado por PentestEval (31 % pipeline completion) y Cyber Defense Benchmark (3,8 % event labeling) como techo actual del estado del arte. Se construye con este fin un sistema multiagente donde un Agente Atacante ejecuta cadenas de ataque siguiendo el framework MITRE ATT&CK con el patrón ReAct sobre LangGraph y un catálogo de 30 herramientas de pentesting, y un Agente Observador opera con información limitada — solo observables de logs HTTP, eventos del sistema y telemetría de red — clasificando independientemente la táctica activa por ventana de 5 s. El entorno se despliega sobre infraestructura Docker con once contenedores en dos redes aisladas y siete escenarios vulnerables (dvwa, mrrobot, dc1, bpent, log4shell, confluence y phpunit). El diseño experimental cubre cuatro ejes: matriz cross-modelo 5×5

atacante $\times$ observer sobre el escenario basic (Eje A, 25 corridas, 19 válidas), generalización a los siete escenarios con el stack ganador (Eje B, 8 corridas), ablación regex-only vs LLM híbrido (Eje C, 2 corridas) y efecto de memoria warm sobre corridas consecutivas (Eje D, 3 corridas). Total: 38 corridas, intervalos de confianza al 95 % por bootstrap percentil (régimen $n=1$ por celda, varianza inter-corrida no caracterizada). Hallazgos principales: (i) el observer alcanza μ macro-F1 = 0,441 sobre basic en la matriz cross-modelo, +190 % sobre el baseline de clase mayoritaria computado de los mismos datos (0,148); DeepSeek-chat es el observer ganador operacional; (ii) generalización a escenarios complejos alcanza μ macro-F1 = 0,429 sobre runs con cadena de ataque al menos parcialmente completada (dc1, bpent-Sonnet, dvwa, phpunit), incluido el caso más robusto dc1 con 6/6 tácticas y mF1 = 0,529; (iii) dos limitaciones centrales — estructurales al enfoque LLM-based, no detalles periféricos — emergen del experimento: el sesgo de frecuencia del LLM atacante GPT-4.1 ante credenciales no canónicas (caso bpent y mrrobot, 1/6 tácticas) y la capacidad insuficiente de modelos free-tier para completar escenarios complejos; (iv) el efecto de memoria warm reduce un 31,6 % las acciones del atacante y mejora monótonicamente el mF1 del observer en un 29,9 % acumulado. La generalización a Active Directory, Windows, exfiltración y movimiento lateral queda como trabajo futuro.

Palabras clave: MITRE ATT&CK, agentes autónomos, Large Language Models, LangGraph, pentesting, detección de intrusiones, sistemas adversariales.

ABSTRACT

The increasing application of Large Language Models (LLMs) to complex reasoning tasks has opened new opportunities in cybersecurity, both for offensive automation and defensive monitoring. This work does NOT propose a new agentic architecture as a contribution per se, but rather empirically measures the accuracy of MITRE ATT&CK tactic inference by an LLM-based observer using network telemetry and system logs in a controlled adversarial setup, within the regime documented by PentestEval (31 % pipeline completion) and Cyber Defense Benchmark (3.8 % event labeling) as the current state-of-the-art ceiling. To this end, a multi-agent system is built where an Attacker Agent executes attack chains following the MITRE ATT&CK framework using the ReAct pattern over LangGraph with a catalog of 30 pentesting tools, and an Observer Agent operates with limited information — only HTTP log observables, system events, and network telemetry — independently classifying the active tactic per 5-second window. The environment is deployed on Docker infrastructure with eleven containers across two isolated networks and seven vulnerable scenarios (dvwa, mrrobot, dc1, bpent, log4shell, confluence, and phpunit). The experimental design spans four axes: 5×5 cross-model attacker × observer matrix on the basic scenario (Axis A, 25 runs, 19 valid), generalization to all seven scenarios with the winning stack (Axis B, 8 runs), regex-only vs hybrid LLM ablation (Axis C, 2 runs), and warm memory effect over consecutive runs (Axis D, 3 runs). Total: 38 runs, 95 % confidence intervals by percentile bootstrap. Main findings: (i) the observer reaches μ macro-F1 = 0.441 on basic in the cross-model matrix, with DeepSeek-chat as the operational winner; (ii) generalization to complex scenarios reaches μ macro-F1 = 0.429 on runs with at least partially completed attack chains (dc1, bpent-Sonnet, dvwa, phpunit), including the most robust case dc1 with 6/6 tactics and mF1 = 0.529; (iii) two Axis B runs (mrrobot and bpent with GPT-4.1, 1/6 tactics) reveal a frequency bias in user enumeration consistent with the LLM memorization literature;

(iv) the warm memory effect reduces attacker actions by 31.6 % and monotonically improves the observer mF1 by 29.9 % cumulative. Generalization to Active Directory, Windows, exfiltration, and lateral movement remains as future work.

Keywords: MITRE ATT&CK, autonomous agents, Large Language Models, LangGraph, pentesting, intrusion detection, adversarial systems.

Contenido

CAPÍTULO 1. INTRODUCCIÓN 8

- 1.1 Contexto y motivación 8
- 1.2 Planteamiento del problema 9
- 1.3 Objetivos 10
 - 1.3.1 Objetivo general. 10
 - 1.3.2 Objetivos específicos. 10
- 1.4 Alcance 11

CAPÍTULO 2. MARCO TEÓRICO 13

- 2.1 El framework MITRE ATT&CK 13
- 2.2 Large Language Models en ciberseguridad 13
- 2.3 LangGraph y el patrón ReAct 14
- 2.4 Sistemas adversariales en ciberseguridad 15

2.5 AI Engineering aplicada al sistema propuesto 15

CAPÍTULO 3. ESTADO DEL ARTE 17

- 3.1 Agentes autónomos para penetration testing 17
- 3.2 Clasificación de tácticas MITRE ATT&CK con LLMs 18
- 3.3 Sistemas adversariales Red Team vs Blue Team 18

CAPÍTULO 4. DISEÑO DEL SISTEMA 20

- 4.1 Arquitectura general 20
- 4.2 Decisiones de diseño 20

4.2.1 Docker en lugar de máquinas virtuales.	20
4.2.2 LangGraph en lugar de alternativas.	21
4.2.3 Loki en lugar de ELK Stack.	22
4.2.4 Selección de modelos de lenguaje.	22
4.2.5 Agentes como procesos host y no como contenedores.	23
4.2.6 Alineación con estándares ISO/IEC y prácticas IEEE.	24
4.3 Agente Atacante	25
4.4 Agente Observador	26
4.4.1 Memoria persistente del observador.	27
4.4.2 Ajuste de umbrales de decisión cost-sensitive.	27
4.4.3 Reestructuración del prompt de clasificación.	28
4.5 Entorno de simulación	28
CAPÍTULO 5. IMPLEMENTACIÓN	30
5.1 Detalles de implementación	30
5.1.1 Repositorio y control de versiones.	30
5.1.2 Stack tecnológico.	30
5.2 Topología de red y arquitectura Docker	32
5.3 Arquitectura lógica de los agentes	33
5.3.1 Agente Atacante — grafo ReAct.	33
5.3.2 Agente Observador — grafo condicional con triaje.	34
5.4 Escenarios de evaluación	35
5.4.1 Escenario básico — DVWA.	35
5.4.2 Escenario Mr. Robot — WordPress CTF.	35
5.4.3 Escenario log4shell — explotación de CVE-2021-44228.	36
5.4.4 Escenario confluence — CVE-2022-26134 OGNL injection.	36
5.4.5 Escenario dc1 — escalada SUID en Drupal.	37
5.4.6 Escenario bpent — boot2root con acceso por wordlist.	37
CAPÍTULO 6. EVALUACIÓN, RESULTADOS Y DISCUSIÓN	39

6.1 Metodología y métricas de evaluación.	39
6.2 Resultados experimentales	40
6.2.1 Resultados — escenario básico.	40
6.2.2 Resultados — escenario Mr. Robot.	41
6.2.3 Comparativa multi-observer (cinco proveedores LLM).	42
6.2.4 Ablation con/sin heurísticas pre-LLM.	44
6.2.5 Matriz de confusión consolidada.	45
6.2.6 Latencia del atacante por táctica.	46
6.3 Discusión	47
6.3.1 Justificación del patrón ReAct.	47
6.3.2 Comparación con baselines clásicos.	48
6.3.3 Análisis de eficiencia y memoria del atacante.	49
6.3.4 Análisis de la dependencia temporal del observador.	51
6.3.5 Análisis de generalización entre escenarios.	52
6.4 Amenazas a la validez del experimento.	53
6.5 Limitaciones del sistema.	55
CAPÍTULO 7. CONCLUSIONES Y TRABAJO FUTURO	58
7.1 Logros y contribuciones del trabajo.	58
7.2 Limitaciones identificadas como aporte.	59
7.3 Trabajo futuro	60
REFERENCIAS	62

CAPÍTULO 1. INTRODUCCIÓN

1.1 Contexto y motivación

Los ataques cibernéticos se han vuelto progresivamente más sofisticados, siguiendo patrones multi-etapa que evolucionan a través de fases distintas antes de alcanzar su objetivo final. El framework MITRE ATT&CK, desarrollado por la corporación MITRE, organiza el comportamiento adversarial en 14 tácticas que van desde el reconocimiento inicial hasta el impacto final sobre los sistemas comprometidos (MITRE Corporation, 2025). Esta taxonomía se ha convertido en el estándar de referencia de la industria para describir, detectar y responder a amenazas persistentes avanzadas.

Paralelamente, los Large Language Models han demostrado capacidades emergentes en tareas de razonamiento complejo, planificación y uso de herramientas. Investigaciones recientes documentan que modelos como Claude Sonnet y GPT-4o logran resultados competitivos en desafíos de penetration testing autónomo, superando enfoques basados en planificación clásica (Nieponice et al., 2025). Esta convergencia entre capacidades de razonamiento de LLMs y necesidades de automatización en ciberseguridad abre una línea de investigación con alto impacto práctico.

La aplicación de LLMs a la seguridad informática exhibe un doble potencial documentado. En el plano ofensivo, trabajos como PentestGPT (Deng et al., 2024) y PentestAgent (Shen et al., 2024) demuestran que los agentes LLM pueden planificar y ejecutar cadenas de ataque autónomas con herramientas reales de pentesting. En el plano defensivo, Srinivas et al. (2025) documentan cómo los LLMs potencian ocho funciones críticas del Security Operations Center, desde el triaje de alertas hasta la respuesta a

incidentes. Zhang et al. (2025), en una revisión sistemática de más de 300 trabajos publicada en Cybersecurity (Springer), confirman que esta aplicación dual constituye la frontera de investigación activa en el área. Sin embargo, la mayoría de los trabajos abordan uno de los dos dominios de forma aislada. Este trabajo los integra en un único sistema adversarial, donde el Agente Atacante demuestra la capacidad ofensiva del LLM y el Agente Observador demuestra su capacidad defensiva, operando de forma concurrente en un entorno controlado con ground truth.

1.2 Planteamiento del problema

El problema que motiva este trabajo surge de la confluencia entre la madurez de los LLMs para tareas de razonamiento complejo y la creciente necesidad de automatización en ciberseguridad. Aunque trabajos recientes demuestran la viabilidad de los LLMs tanto para tareas ofensivas como defensivas, estas capacidades han sido exploradas en su mayoría de forma separada. La ausencia de marcos integrados que evalúen simultáneamente ambas dimensiones limita la comprensión de cómo los LLMs se desempeñan en condiciones adversariales reales.

Deng et al. (2024) identifican en PentestGPT, evaluado en USENIX Security 2024, que los LLMs son competentes en sub-tareas individuales de pentesting pero pierden el contexto global en compromisos de largo horizonte, un déficit que cualquier agente de seguridad autónomo debe superar. Daniel et al. (2024) encuentran que los LLMs proveen clasificaciones explicables de tácticas MITRE, pero que la contextualización temporal de secuencias de observables sigue siendo un desafío: un mismo observable puede corresponder a múltiples tácticas dependiendo del estado del ataque. Estas evidencias señalan que el desempeño de los LLMs en tareas de seguridad complejas requiere validación en entornos controlados que permitan medir resultados de forma objetiva.

Este trabajo construye un banco de pruebas adversarial controlado que permite estudiar empíricamente el comportamiento de agentes LLM en contextos de ciberseguridad ofensiva y defensiva. La pregunta de investigación central es: ¿en qué medida puede un sistema multiagente basado en LLMs simular cadenas de ataque reales, observar el comportamiento adversarial mediante telemetría de red, e inferir automáticamente las tácticas del atacante en un entorno de laboratorio controlado? Esta pregunta amplía el foco habitual de clasificación táctica hacia la evaluación sistémica del bucle completo ataque–observación–interpretación, siguiendo el enfoque de measurement paper propuesto por He et al. (2023) [18] para la evaluación empírica de sistemas LLM en ciberseguridad.

1.3 Objetivos

1.3.1 Objetivo general.

Desarrollar un sistema adversarial basado en agentes autónomos donde un Agente Atacante ejecute cadenas de ataque completas siguiendo el framework MITRE ATT&CK, y un Agente Observador clasifique automáticamente la táctica en progreso mediante análisis de logs, tráfico de red y eventos del sistema, utilizando LLMs para razonamiento y orquestación.

1.3.2 Objetivos específicos.

Diseñar e implementar un Agente Atacante autónomo capaz de ejecutar cadenas de ataque que progresen a través de múltiples tácticas MITRE ATT&CK utilizando herramientas reales de pentesting.

Desarrollar un Agente Observador que opere con información limitada, simulando la perspectiva de un sistema de monitoreo real, e identifique la táctica activa mediante clasificación basada en LLMs.

Implementar un entorno de simulación controlado con infraestructura vulnerable donde el Agente Atacante pueda operar y el Agente Observador recolecte observables sin acceso directo a las acciones del atacante.

Evaluar la precisión del sistema de clasificación mediante comparación entre el ground truth registrado por el atacante y las clasificaciones emitidas por el observador.

1.4 Alcance

El presente trabajo cubre seis tácticas centrales del kill chain como espacio de ejecución del Agente Atacante; el espacio de salida del Agente Observador comprende las catorce tácticas MITRE ATT&CK Enterprise (incluyendo la clase "none" para ventanas sin actividad maliciosa). El sistema cubre MITRE ATT&CK Enterprise — Reconnaissance (TA0043), Initial Access (TA0001), Execution (TA0002), Discovery (TA0007), Credential Access (TA0006) y Privilege Escalation (TA0004) — sobre siete escenarios adversariales con perfiles distintos: dvwa y mrrobot (CTF clásicos WordPress/PHP), dc1 y bpent (boot2root con escalada SUID), log4shell (CVE-2021-44228), confluence (CVE-2022-26134, OGNL injection) y phpunit (CVE-2017-9841, eval-stdin RCE). Los tres últimos son escenarios pre-auth donde Initial Access y Credential Access no aplican como tácticas separadas porque la explotación directa otorga ejecución remota; el agente cubre Reconnaissance, Execution y Discovery sobre estos. La evaluación cuantitativa se realiza sobre las clasificaciones del observador comparadas contra el ground truth registrado por el atacante; bajo la restricción metodológica de sincronización 1:N táctica/ventana documentada en §4.3 (default desde mayo 2026), la clasificación del observer es single-label / multi-class por ventana. Las métricas multi-label permanecen implementadas en `src/evaluation/metrics.py` por compatibilidad con corridas previas y como salvaguarda ante regímenes futuros multi-tactic per window. Las tácticas restantes del kill chain (Persistence, Lateral Movement, Defense Evasion, Collection, Command and Control, Exfiltration,

Impact, Resource Development) y la generación de informes de remediación corresponden a trabajo futuro.

CAPÍTULO 2. MARCO TEÓRICO

2.1 El framework MITRE ATT&CK

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) es una base de conocimiento globalmente accesible que estructura el comportamiento adversarial en matrices de tácticas y técnicas derivadas de observaciones de ataques reales (MITRE Corporation, 2025; Strom et al., 2018). La matriz Enterprise comprende 14 tácticas que representan los objetivos tácticos que un adversario puede perseguir durante un ataque, desde la recopilación de información inicial (Reconnaissance) hasta la destrucción o manipulación de datos (Impact).

Cada táctica contiene un conjunto de técnicas que describen cómo un adversario puede lograr ese objetivo táctico, y cada técnica incluye sub-técnicas con procedimientos específicos. Esta jerarquía permite clasificar el comportamiento adversarial en tres niveles de granularidad: táctica (qué objetivo persigue), técnica (cómo lo logra) y sub-técnica (variante específica del método). Para este trabajo, el nivel de clasificación relevante es el táctico, pues provee información estratégica accionable para defensores.

Referencia industrial complementaria. Las evaluaciones MITRE ATT&CK Enterprise 2025 reportadas por Forrester [9] (post comercial, no peer-reviewed pero referencia industrial actualizada) documentan que los SIEMs comerciales efectivos (CrowdStrike, Cybereason) consolidan alertas de técnicas individuales en casos únicos a nivel táctico, validando empíricamente que la clasificación táctica agregada (no por técnica) es la salida valiosa para defensores. La comparación directa en F1 contra estos productos cerrados no se realiza por inaccesibilidad del código, pero el alineamiento de granularidad táctica de este trabajo con la industria es consistente.

2.2 Large Language Models en ciberseguridad

Los Large Language Models son modelos de aprendizaje profundo entrenados sobre corpus masivos de texto que exhiben capacidades emergentes de razonamiento, planificación y síntesis de información. En el contexto de ciberseguridad, Srinivas et al. (2025) mapean sistemáticamente la aplicación de LLMs a ocho funciones críticas de Security Operations Centers: resumen de logs, triaje de alertas, inteligencia de amenazas y respuesta a incidentes, entre otras. Una revisión sistemática de 105 estudios peer-reviewed seleccionados de un corpus inicial superior a 600 trabajos confirma que los LLMs mejoran la precisión de detección y reducen la latencia de respuesta comparados con enfoques tradicionales.

Vinay (2025) presenta una taxonomía de cinco generaciones en la evolución de IA agéntica en ciberseguridad, desde razonadores LLM de texto simple hasta sistemas multi-agente con pipelines autónomos. El sistema propuesto en este trabajo corresponde a la tercera generación de esta taxonomía: agentes aumentados con herramientas que ejecutan acciones en entornos reales. Esta ubicación en la taxonomía define tanto las capacidades esperadas como las limitaciones conocidas del enfoque.

Zhang et al. (2025) sistematizan esta aplicación dual en una revisión sistemática (105 estudios seleccionados de un corpus inicial superior a 600 trabajos) publicada en *Cybersecurity* (Springer), documentando el uso de LLMs tanto en detección de intrusiones, análisis de amenazas y respuesta a incidentes en el plano defensivo, como en reconocimiento, explotación y evaluación de vulnerabilidades en el plano ofensivo. Divakaran y Peddinti (2024) argumentan que los LLMs abren oportunidades cualitativamente nuevas respecto a los enfoques previos de aprendizaje automático, específicamente a través de la comprensión semántica de logs y reportes de amenazas que los clasificadores numéricos no pueden realizar. Este trabajo explora empíricamente esa capacidad en un entorno adversarial controlado.

2.3 LangGraph y el patrón ReAct

LangGraph es un framework de orquestación de agentes (LangChain Inc., 2025) que representa flujos de trabajo como grafos de estado dirigidos. A diferencia de LangChain, que está optimizado para pipelines lineales de llamadas a LLMs, LangGraph introduce una capa formal de orquestación que permite ciclos de razonamiento, colaboración multi-agente y ejecución dirigida por eventos. Esta distinción es fundamental para implementar agentes que deben adaptar su comportamiento en función de resultados intermedios.

El patrón ReAct (Reasoning + Acting) es un esquema de interacción en el que el LLM alterna entre razonamiento sobre el estado actual, selección de una acción a ejecutar, y observación del resultado para informar el siguiente ciclo de razonamiento. Este patrón ha demostrado superar enfoques de solo razonamiento o solo actuación en benchmarks de tareas que requieren interacción con entornos externos (Yao et al., 2023). Para el Agente Atacante, el patrón ReAct permite que el LLM adapte su estrategia de ataque en función de lo que descubre en cada paso.

2.4 Sistemas adversariales en ciberseguridad

Los sistemas adversariales en ciberseguridad modelan la interacción entre un atacante y un defensor como un proceso dinámico de aprendizaje mutuo. Huang et al. (2025) presentan el framework RvB (Red vs Blue), donde equipos adversariales iteran para mejorar capacidades ofensivas y defensivas simultáneamente, logrando Defense Success Rates del 90% en hardening de código dinámico mientras mantiene near-zero False Positive Rates. La clave de este enfoque es que el defensor aprende de las acciones del atacante, pero sin comunicación directa durante la operación.

Este principio de aislamiento de información es central en el diseño del sistema propuesto: el Agente Observador no tiene acceso a las decisiones o el estado interno del

Agente Atacante, simulando las condiciones reales que enfrenta un analista de seguridad que solo puede observar los efectos del ataque en los sistemas monitoreados.

2.5 AI Engineering aplicada al sistema propuesto

El sistema propuesto adopta deliberadamente cuatro principios de AI Engineering — la disciplina que aborda el ciclo de vida completo de sistemas basados en modelos de aprendizaje, desde datos hasta despliegue — recomendados por la literatura reciente y por estándares emergentes para sistemas de IA (ISO/IEC 23053:2022 Framework for AI Systems Using Machine Learning).

Primero, agentes basados en patrones de diseño documentados. El Agente Atacante implementa el patrón ReAct (Reasoning + Acting) propuesto por Yao et al. (2023) [10], donde el LLM alterna entre razonamiento sobre el estado actual, selección de una acción a ejecutar, y observación del resultado. El Agente Observador implementa el patrón Triage → Investigate → Classify → Escalate descrito por Vinay (2025) [5] para pipelines SOC con LLM, separando explícitamente la fase de filtrado pre-LLM (sin invocación) de la fase de clasificación (con LLM). La adopción de estos patrones se fundamenta empíricamente: ReAct supera Chain-of-Thought y Act-only en tareas con interacción de entornos (Yao 2023); el patrón Triage reduce los falsos positivos en clases minoritarias del kill chain (validado en el experimento de ablation, sección 5.5.5).

Segundo, multi-proveedor de modelos como capa intercambiable. La capa de proveedores LLM se generalizó a seis backends (OpenAI, Anthropic, Google, Groq, OpenRouter, Cerebras) intercambiables vía variables de entorno. Esta abstracción permite la matriz comparativa empírica reportada en la sección 6.2.3 sin re-implementar la lógica del agente. La literatura sobre evaluación en seguridad (AthenaBench, Liu et al. 2025 [29]) demanda

exactamente este tipo de matrices comparativas — no afirmaciones sobre un solo modelo — para sostener claims de generalización.

Tercero, reproducibilidad como requisito de primera clase. Cada corrida fija seed=42 (donde el proveedor lo soporta), temperature por rol (0.2 atacante, 0.0 observador) y persiste metadata en EvaluationReport (modelo, hash de commit, timestamp ISO, escenario, parámetros). Estas prácticas se alinean con ISO/IEC TR 24028:2020 (Trustworthiness in AI) sobre transparencia y trazabilidad, y con ISO/IEC TS 4213:2022 sobre evaluación de clasificadores ML.

Cuarto, prompt engineering como palanca de primera magnitud. La fase exploratoria temprana del proyecto sobre mrrobot (pre-anti-cheating, régimen comparable interno, §6.2.2) registra una mejora aproximada de +0,25 absoluto en Macro F1 sobre el mismo modelo entre prompts iniciales y prompts post-mejora (continuidad de fase post-compromiso y desambiguación priv_esc/discovery), magnitud comparable o mayor a las diferencias entre proveedores en la matriz cross-modelo del Eje A. Estas cifras corresponden a un régimen pre-anti-cheating y no son directamente comparables con los resultados del Eje B; el orden de magnitud del salto, sin embargo, motiva el tratamiento del prompt como palanca de primera magnitud. La estructura del prompt del observer se versionó como código (no como texto libre), lo que permite ablation rigurosa sobre cada bloque del prompt en trabajo futuro.

CAPÍTULO 3. ESTADO DEL ARTE

Nota: varias referencias citadas en este capítulo y a lo largo del documento corresponden a preprints disponibles en arXiv o TechRxiv (referencias [1], [3], [4], [5], [15], [27]) que no han pasado revisión por pares al cierre de este trabajo. Se citan por reflejar el estado de avance reciente del campo, pero las afirmaciones cuantitativas extraídas de ellas se reportan como tendencias y no como resultados consolidados.

3.1 Agentes autónomos para penetration testing

La automatización de penetration testing mediante LLMs ha avanzado rápidamente en los últimos años. Shen et al. (2024) proponen PentestAgent, un framework que utiliza colaboración multi-agente para automatizar reconnaissance, análisis de vulnerabilidades y explotación mediante Retrieval Augmented Generation. Los resultados demuestran rendimiento superior en completitud de tareas y eficiencia, reduciendo significativamente la intervención manual.

Nieponice et al. (2025) presentan ARACNE, un agente de pentesting autónomo basado en LLMs para servicios SSH que implementa una arquitectura modular con cuatro componentes: planificador, intérprete, resumidor y agente central. ARACNE logra una tasa de éxito del 60% (cifra reportada en preprint, sin revisión por pares completa al cierre de este trabajo) contra el defensor autónomo ShellLM y 57.58% en los desafíos Over The Wire Bandit CTF, con un promedio de menos de 5 acciones por objetivo. Este trabajo evidencia que los LLMs pueden ejecutar cadenas de ataque coherentes con herramientas reales, aunque las limitaciones en razonamiento de largo horizonte persisten como desafío abierto.

Deng et al. (2024) presentan PentestGPT, evaluado en USENIX Security 2024, donde demuestran que los LLMs logran una mejora del 228.6% en completitud de tareas de pentesting respecto al baseline, pero identifican la pérdida de contexto en horizontes largos

como el principal desafío abierto. Este hallazgo motiva el diseño de agentes atacantes con estado persistente explícito, como el implementado en este trabajo mediante LangGraph. Huang y Zhu (2024) extienden el paradigma integrando un módulo de remediación, evidenciando el potencial de los LLMs para generar output estructurado sobre las vulnerabilidades descubiertas.

A diferencia de estos trabajos, que se centran en la dimensión ofensiva de forma aislada, este proyecto integra un agente atacante con un observador independiente que opera en el mismo entorno controlado. Esta arquitectura permite demostrar simultáneamente la capacidad ofensiva y defensiva del LLM dentro de un marco con ground truth verificable.

3.2 Clasificación de tácticas MITRE ATT&CK con LLMs

Daniel et al. (2024, preprint arXiv:2412.10978) investigan la capacidad de LLMs (ChatGPT, Claude, Gemini) para etiquetar reglas de detección de red con tácticas y técnicas MITRE ATT&CK, experimentando con 973 reglas Snort. Los resultados indican que aunque los LLMs proveen mapeos iniciales explicables, escalables y eficientes, los modelos de Machine Learning tradicionales los superan consistentemente en precisión, recall y F1-score. Sin embargo, la ventaja de los LLMs reside en su capacidad para razonar sobre contexto y proveer justificaciones interpretables, cualidad que es particularmente valiosa cuando el analista necesita entender el por qué de una clasificación.

Las evaluaciones MITRE ATT&CK 2025 reportadas por Forrester Research revelan que proveedores efectivos de detección consolidan alertas en casos únicos en lugar de generar cientos de alertas dispersas, validando el valor de la clasificación a nivel táctico sobre alertas granulares de técnicas individuales. Este hallazgo refuerza la relevancia del enfoque de clasificación de tácticas adoptado en este trabajo.

3.3 Sistemas adversariales Red Team vs Blue Team

Huang et al. (2026, preprint arXiv:2601.19726) reportan que la adaptación adversarial iterativa entre equipos Red y Blue alcanza Defense Success Rates del 90% en hardening de código dinámico, estableciendo que el objetivo primario de los sistemas adversariales es minimizar la incertidumbre epistémica del defensor mediante agentes ofensivos que funcionan como herramientas diagnósticas. Este principio fundamenta la arquitectura propuesta, donde el Agente Atacante no solo ejecuta ataques sino que también genera el ground truth necesario para evaluar al Agente Observador.

Vinay (2025) identifica como desafíos no resueltos en sistemas adversariales agénticos la validación de respuestas, correctitud en el uso de herramientas, coordinación multi-agente y razonamiento de largo horizonte. El diseño del sistema propuesto aborda explícitamente el primero mediante el registro automático de ground truth, y el segundo mediante herramientas de pentesting probadas (nmap, hydra, sqlmap) con salidas estructuradas.

CAPÍTULO 4. DISEÑO DEL SISTEMA

4.1 Arquitectura general

El sistema se organiza en tres capas: infraestructura, agentes y orquestación. La capa de infraestructura provee el entorno de simulación controlado mediante Docker. La capa de agentes contiene el Agente Atacante y el Agente Observador, que operan concurrentemente sin comunicación directa. La capa de orquestación coordina la ejecución de ambos agentes y gestiona la comparación de resultados al finalizar la simulación.

El flujo de información es unidireccional e indirecto: el Agente Atacante ejecuta comandos en el container atacante, estos generan tráfico hacia el objetivo vulnerable, el objetivo genera logs como efecto de las acciones del atacante, el stack de logging recopila esos logs, y el Agente Observador los consulta para emitir su clasificación. En ningún punto el Agente Observador accede a información sobre las decisiones o el estado interno del Agente Atacante.

4.2 Decisiones de diseño

4.2.1 Docker en lugar de máquinas virtuales.

El entorno de simulación se implementa sobre contenedores Docker en lugar de máquinas virtuales tradicionales (VirtualBox, VMware) por tres razones fundamentales. En primer lugar, el rendimiento: Sobieraj y Kotyński (2024) demuestran que los contenedores incurren significativamente menos overhead que las máquinas virtuales en benchmarks de CPU, I/O y red, mientras estudios de adopción empresarial reportan que las VMs consumen 2-4 veces más memoria que sus contrapartes contenerizadas para cargas de trabajo equivalentes. Para un proyecto que requiere ejecutar cinco servicios simultáneos en un equipo con 16 GB de RAM, esta diferencia es determinante.

En segundo lugar, la reproducibilidad: la infraestructura Docker se describe completamente en un archivo `docker-compose.yml` con versiones fijadas de todas las imágenes, garantizando que el entorno de simulación sea idéntico en cualquier máquina donde se despliegue. En tercer lugar, la disponibilidad de imágenes pre-configuradas: la imagen `vulnerables/web-dvwa` provee una aplicación web con todas las vulnerabilidades habilitadas por defecto, eliminando la necesidad de configuración manual.

4.2.2 LangGraph en lugar de alternativas.

La selección de LangGraph como framework de orquestación de agentes se realizó tras evaluar cuatro alternativas: LangChain simple, CrewAI, AutoGen y n8n. LangChain simple, aunque provee las abstracciones base de herramientas y memoria, no soporta grafos de estado con ciclos condicionales, que son necesarios para implementar el loop ReAct del Agente Atacante donde la decisión de continuar ejecutando herramientas o avanzar de táctica depende del estado actual.

CrewAI y AutoGen están optimizados para sistemas multi-agente con comunicación directa entre agentes mediante mensajes, lo que no se alinea con el requisito de aislamiento de información entre el atacante y el observador. n8n es una herramienta de automatización de flujos de trabajo orientada a integraciones empresariales sin código, sin las capacidades de manejo de estado de grafos necesarias para implementar razonamiento agéntico iterativo. Una comparación sistemática de toolchains agénticos disponible como preprint en TechRxiv (Sapkota et al., 2025, no peer-reviewed al cierre) [15] confirma que LangGraph introduce una capa formal de orquestación que representa flujos como grafos de estado con estado dirigido, habilitando razonamiento ciclico y ejecución dirigida por eventos, capacidades ausentes en los frameworks alternativos evaluados.

4.2.3 Loki en lugar de ELK Stack.

El sistema de logging centralizado utiliza Grafana Loki en lugar de ELK Stack (Elasticsearch, Logstash, Kibana) por razones de eficiencia de recursos. Loki indexa únicamente metadatos de los streams de logs (etiquetas de container, timestamp), mientras Elasticsearch realiza indexación de texto completo. Esta diferencia arquitectónica se traduce en que el índice de Loki es menor al 1% del volumen de logs, mientras el índice de Elasticsearch puede superar 2-10 veces el volumen de datos originales. En términos de memoria RAM, Loki opera dentro de 500 MB frente a los 3-4 GB mínimos requeridos por un stack ELK funcional, diferencia crítica en el contexto de recursos de hardware disponibles.

4.2.4 Selección de modelos de lenguaje.

La selección del modelo de lenguaje para cada agente consideró tres criterios: calidad de razonamiento para flujos agénticos, latencia de respuesta e implicaciones operacionales de los límites de tasa (tokens por minuto, TPM) impuestos por la API del proveedor. La arquitectura del sistema permite intercambiar el proveedor mediante una variable de configuración, garantizando independencia del ecosistema de un único proveedor.

Para el Agente Atacante se seleccionó GPT-4.1 de OpenAI. Este modelo fue diseñado específicamente para flujos agénticos de múltiples pasos, con una mejora de 21,4 puntos absolutos sobre GPT-4o en el benchmark SWE-bench Verified (54,6% vs 33,2%) y una mejora de 20 puntos en razonamiento sobre contextos largos (OpenAI, 2025). El atacante debe mantener estado a lo largo de múltiples iteraciones ReAct, adaptar su estrategia ante resultados inesperados y encadenar herramientas de forma coherente: estas características justifican el uso del modelo de mayor capacidad.

Para el Agente Observador la versión inicial seleccionó gpt-4o-mini de OpenAI por su límite holgado de tokens por minuto, pero la evolución posterior del proyecto generalizó la

capa de proveedores a seis backends intercambiables (OpenAI, Anthropic, Google, Groq, OpenRouter, Cerebras) seleccionables vía la variable de entorno `OBSERVER_PROVIDER`. Esta generalización permitió la matriz comparativa empírica reportada en la sección 6.2.3, donde se documenta el trade-off latencia/calidad: en el Eje A sobre basic, DeepSeek-chat es el observer ganador operacional (μ macro-F1 = 0,479; latencia avg 7,94 s, sostenido en todos los escenarios del Eje B); GPT-4.1-mini lidera en latencia (5,66 s) y Cerebras Qwen3 235B vía free tier opera en latencias muy superiores (~114 s avg, inviable para SOC en línea con ventanas de 5 s) sin liderar en mF1 absoluto. El stack final del Eje B fija GPT-4.1 atacante + DeepSeek-chat observer.

La selección de proveedores distintos para cada agente demuestra la interoperabilidad del diseño: el Agente Atacante usa OpenAI y el proveedor del Agente Observador es configurable via variable de entorno `LLM_PROVIDER`.

4.2.5 Agentes como procesos host y no como contenedores.

Una decisión arquitectónica deliberada del sistema es que los dos agentes — Atacante y Observador — corren como procesos Python en el sistema host y no dentro de contenedores Docker. La capa Docker aloja únicamente los componentes del entorno simulado: el contenedor atacante con las herramientas de pentesting (Kali Linux), el contenedor objetivo vulnerable, y el stack de logging (Promtail, Loki, Grafana). Los agentes orquestan estos contenedores desde fuera mediante el SDK de Docker (docker exec para enviar comandos al contenedor atacante) y el cliente HTTP de Loki para consultar logs. Esta decisión responde a cinco motivos:

(1) Separación entre control plane y data plane. Los agentes representan el plano de control: toman decisiones, mantienen estado, planifican acciones, persisten memoria entre corridas. Los contenedores representan el plano de datos: ejecutan herramientas, alojan el target,

generan logs. Mantener estos dos planos en runtimes distintos es una decisión arquitectónica estándar que simplifica el modelo mental, facilita el debugging independiente y reduce el acoplamiento entre componentes.

(2) Realismo del modelo de amenaza. En una operación real, el pentester humano corre desde su máquina y los comandos llegan al endpoint comprometido a través de un canal C2 (SSH, reverse shell, webshell). Análogamente, el analista SOC corre desde su sistema de monitoreo (SIEM), no desde dentro de los servidores monitoreados. La arquitectura host-agent + container-target replica esa separación física entre el actor y el sistema bajo observación, alineándose con el principio de Realismo Operacional definido en la planificación del proyecto.

(3) Independencia de runtimes. El contenedor atacante corre Kali Linux con nmap, hydra, sqlmap y otras herramientas de pentesting. Los agentes corren Python 3.12 con LangGraph y los SDKs de los proveedores LLM. Estos dos runtimes son ortogonales: meter Python y dependencias de OpenAI/Anthropic en la imagen de Kali contamina la herramienta de pentest con un stack distinto, obliga a mantener dos cadenas de dependencias en una sola imagen y aumenta el tamaño del contenedor sin aportar capacidad nueva.

(4) Acceso simple a APIs externas y al stack de logging. Los agentes consumen los proveedores LLM via HTTPS y consultan Loki en localhost:3100. Como procesos host, los dos accesos son directos. Si los agentes estuvieran dentro de Docker tendrían que estar en la red correcta (o en ambas redes), añadir reglas de salida hacia las APIs LLM, configurar DNS para resolver dominios externos, etc. Esta complejidad es incidental al objetivo del experimento y no aporta valor metodológico.

(5) Iteración rápida durante desarrollo. Un cambio en el prompt o en la lógica del agente se prueba inmediatamente con ``python -m src.main``. Si los agentes estuvieran empaquetados en

contenedores, cada cambio requeriría rebuild + restart, multiplicando la latencia de iteración por un orden de magnitud durante el desarrollo. Para un proyecto académico con experimentación intensiva, esta diferencia es determinante en el progreso del trabajo.

Como contraste, sí se justifica containerizar los agentes en un despliegue de producción donde la inmutabilidad y la reproducibilidad del entorno de ejecución sean requisitos. Para el alcance experimental de este trabajo, la portabilidad del agente como proceso host (acompañado del archivo pyproject.toml con dependencias fijadas) es suficiente.

4.2.6 Alineación con estándares ISO/IEC y prácticas IEEE.

El diseño y la evaluación del sistema se alinean con cuatro estándares internacionales relevantes para sistemas de IA aplicada a clasificación. ISO/IEC TS 4213:2022 (Assessment of machine learning classification performance) define el protocolo de evaluación de clasificadores ML y prescribe el reporte de precision, recall y F1-score por clase con promedios micro y macro: este es exactamente el conjunto de métricas que se reporta en la sección 5.5 y que se implementa en el módulo `src/evaluation/metrics.py` siguiendo Sokolova & Lapalme (2009). ISO/IEC 22989:2022 (Artificial intelligence concepts and terminology) provee el vocabulario formal para describir los agentes (planning agent, perception agent, autonomous system) que se usa de manera consistente en este documento. ISO/IEC 23053:2022 (Framework for AI Systems Using Machine Learning) establece el ciclo de vida — datos, modelo, evaluación, despliegue — que estructura el Capítulo 4 (Diseño) y el Capítulo 5 (Implementación y Resultados). ISO/IEC TR 24028:2020 (Trustworthiness in AI) discute requerimientos de trazabilidad, transparencia y reproducibilidad: el sistema los aborda mediante (a) `seed=42` fijado en la generación, (b) registro de metadata por corrida (modelo, hash de commit, timestamp) en `EvaluationReport`, y (c) ground truth automático capturado por el atacante.

Adicionalmente, en lo que respecta a ingeniería de software, el proyecto adopta prácticas alineadas con ISO/IEC/IEEE 12207:2017 (Software life cycle processes) en el desarrollo iterativo del sistema, con control de versiones git y pruebas unitarias por cada componente.

4.3 Agente Atacante

El Agente Atacante implementa el patrón ReAct mediante un grafo LangGraph con cinco nodos: `plan_tactic`, `execute_tools`, `validate_result`, `check_objective` y `advance_tactic`. El nodo `plan_tactic` invoca al LLM con el contexto de la táctica actual y los datos recopilados en pasos anteriores, produciendo un plan de acción expresado como llamadas a herramientas. El nodo `execute_tools` ejecuta esas herramientas en el container atacante via Docker SDK y recopila los resultados. El nodo `validate_result` invoca nuevamente al LLM para que analice los resultados y decida si continuar con más acciones dentro de la misma táctica o declararla completa. El nodo `advance_tactic` gestiona la transición entre tácticas.

A la fecha de esta entrega el agente cuenta con un catálogo de treinta herramientas registradas como LangChain tools, organizadas en cinco categorías: reconocimiento (`run_nmap`, `run_nikto`, `run_gobuster`, `run_dirsearch`, `run_wpscan`, `run_whatweb`, `run_enum4linux`, `run_smbclient`, `run_dns_enum`, `run_searchsploit`), explotación (`run_sqlmap`, `run_hydra`, `run_hydra_http_form`, `run_http_session`, `run_curl`, `run_command`, `run_ssh_exec`, `run_ftp`, `run_file_upload`), payloads (`run_msfvenom`, `write_exploit_file`, `start_reverse_listener`, `serve_http`), escalada de privilegios (`run_priv_esc_enum`, `run_linpeas`, `run_web_shell`), y utilitarios (`run_john`, `decode_string`, `run_spider`, `run_gobuster_recursive`). Cada ejecución de herramienta registra automáticamente un `ActionRecord` con táctica, técnica, comando, timestamp y resultado, constituyendo el ground truth del experimento.

La elección del patrón ReAct responde a características intrínsecas del dominio de pentesting. Un pentest es un proceso de decisión secuencial: cada herramienta modifica el estado del

objetivo y genera observaciones que condicionan la siguiente acción. Un agente puramente predictivo (sin loop observe → think → act) no puede responder a hallazgos imprevistos como puertos cerrados, credenciales rechazadas o respuestas anómalas; debe re-planificar. Yao et al. (2023) [10] muestran empíricamente que este patrón supera arquitecturas secuenciales planas (chain-of-thought) en tareas de razonamiento sobre estado externo. El presente sistema implementa ReAct con la restricción adicional de validators code-based en lugar del honor system propio del trabajo original. Restricción metodológica de sincronización atacante-ventana: desde mayo de 2026 el agente atacante incorpora un mecanismo de sincronización con la ventana de observación del observer (configurable mediante settings.attacker_tactic_per_window, activado por defecto). Cuando el atacante transita a una nueva táctica, el nodo execute_tools invoca `_wait_for_next_window_boundary()` y bloquea la ejecución hasta el inicio de la siguiente ventana del observer (calculada como $\text{simulation_start} + \text{interval} \times \lceil (\text{now} - \text{simulation_start}) / \text{interval} \rceil$). Esto fuerza una correspondencia 1:N entre táctica del atacante y ventanas observables: una táctica puede ocupar $N \geq 1$ ventanas consecutivas, pero ninguna ventana contiene más de una táctica del atacante. La motivación es metodológica, no operacional: separar temporalmente las tácticas en el ground truth simplifica la interpretación del strict_accuracy y de la matriz de confusión al convertir la clasificación del observer de multi-label genuino a single-label / multi-class por ventana, sin alterar la dinámica del ataque (replans, razonamiento del LLM y validators code-based no se ven afectados — solo el primer docker exec de cada nueva táctica). El overhead temporal acumulado por estos sleeps de sincronización se reporta en metadata por corrida (campo tactic_wait_for_window) y se discute en §6.4 como restricción deliberada de validez interna. Principio anti-cheating del atacante (referenciado por §6.4): el prompt del Agente Atacante incluye explícitamente la prohibición de invocar credenciales, paths o procedimientos memorizados de writeups

públicos del corpus de pre-entrenamiento. Cualquier acción debe estar justificada por evidencia recolectada en la corrida actual (output de tools, no conocimiento previo del LLM). Los validators code-based del nodo `validate_result` complementan este principio rechazando como evidencia cualquier salida fabricada por `echo`, `printf` o `python -c print()` —operativamente, el atacante no puede declarar éxito sin output observable de la herramienta real. Este mecanismo conjunto es lo que en este trabajo se denomina módulo anti-cheating; su impacto cuantificado sobre el desempeño aparente del agente se reporta en §6.2.2.

4.4 Agente Observador

El Agente Observador implementa un grafo `LangGraph` de seis nodos con ramificación condicional y un loop de refinamiento: `collect_logs` → `triage_anomalies` → `detect_anomalies` → `classify_tactic` → `refine_analysis` (loop, máximo 2 iteraciones si confianza adaptativa no se alcanza) → `generate_recommendation`.

El nodo `collect_logs` consulta la API HTTP de Loki mediante `LogQL` y recupera hasta 1500 entradas de log dentro de una ventana de 5 segundos (parámetros calibrados empíricamente sobre la base de Bhuyan et al. (2014) [24] para reducir el efecto de drowning de eventos breves bajo el volumen de logs de scanner; ver sección 4.4.3). Aplica un algoritmo de deduplicación con cola reciente: los logs anteriores al último minuto de la ventana se agregan por patrón (IP, método, ruta base, status HTTP) —colapsando miles de requests repetitivos de herramientas como nikto en pocas líneas—, mientras que las 50 entradas más recientes se envían completas para preservar la señal de la táctica actual. Este procesamiento acota el output a aproximadamente 200-600 líneas independientemente del volumen total.

El nodo `triage_anomalies` aplica heurísticas deterministas sin invocar al LLM: si la cantidad de logs que contienen palabras clave de ataque (`scan`, `login`, `exec`, `shell`, etc.) no supera 3 entradas o el 10% del total, el ciclo termina sin costo de inferencia. Este nodo

implementa el paso 'Triage' del patrón Triage → Investigate → Classify documentado por Vinay (2025), asegurando que el LLM solo se invoca cuando hay evidencia suficiente de actividad anómala.

El nodo `classify_tactic` es la única invocación al LLM por ciclo. Construye un prompt que incluye: las señales pre-calculadas del nodo anterior, el resumen deduplicado de logs, el historial de las últimas ocho clasificaciones como contexto de progresión del ataque, y las definiciones de las 14 tácticas MITRE ATT&CK con sus observables característicos. El LLM responde con un JSON estructurado que incluye táctica clasificada, identificador MITRE, nivel de confianza (0.0-1.0), evidencia citada de los logs, razonamiento y recomendación para el defensor. Si la confianza es menor a 0.65, el nodo `refine_analysis` genera una vista forense alternativa —agrupando actividad por IP y priorizando las entradas con mayor densidad de keywords de ataque— y reenvía al nodo `classify_tactic` para un segundo intento.

El nodo `detect_anomalies` construye perfiles de IP a partir del HTTP status code sin invocar al LLM: detecta `webshell_execution` (HTTP 200 en rutas con `cmd=`), `login_success` (HTTP 302 en `wp-login.php POST`), `brute_force_4xx` (alta densidad de 4xx) y `scan_burst` (alta cardinalidad de paths nuevos en ventana corta). Estas señales se inyectan directamente en el prompt del nodo siguiente como pre-clasificación determinística, lo que reduce la carga cognitiva del LLM al limitar su tarea a ratificar o refinar señales ya identificadas en lugar de descubrirlas desde cero. Procesamiento secuencial del observer (sin saltos de ventana): el loop principal del observador en `src/main.py` incrementa el cursor `last_end` exactamente en `interval_delta` por iteración, con `interval_delta = poll_interval` (5 s). Cuando la latencia LLM excede el intervalo de polling, las ventanas se acumulan en un backlog lógico — pero NO se descartan ni se saltan. Una flush phase explícita procesa todas las ventanas pendientes hasta cubrir el último evento del atacante más un intervalo de seguridad: la prioridad operativa es cobertura completa del eje temporal, no latencia de respuesta. Esta decisión se justifica por el

régimen forense en el que opera el sistema (análisis post-hoc de logs, no alertamiento en tiempo real) y porque saltar ventanas introduciría sesgo no observado en la matriz de confusión: las ventanas saltadas serían sistemáticamente las que coinciden con tácticas rápidas del atacante, sub-representando dichas clases. El backlog ratio reportado en §6.5 ($\text{latencia} / \text{polling} - 1.0$) es métrica diagnóstica del retraso acumulado, no de ventanas perdidas.

4.4.1 Memoria persistente del observador.

A partir de la fase actual, el observador integra una memoria persistente por fingerprint del patrón de tráfico (containers activos, distribución de métodos HTTP y de status codes top-3). Para cada fingerprint, el sistema mantiene la distribución empírica de tácticas observadas en corridas anteriores y la secuencia típica de la kill chain. Esta información se inyecta en el prompt de clasificación como prior bayesiano con la instrucción explícita de tratarlo como sugerencia, no como certeza: si la evidencia del log actual contradice el prior, prevalece la evidencia. La justificación académica se apoya en NIST SP 800-94 [21], que recomienda baselines per-asset en SIEMs efectivos, y en Vinay (2025) [5], que reporta reducciones medibles en falsos positivos al integrar memoria histórica en clasificadores LLM (Vinay 2025 reporta el principio sin que este trabajo haya verificado la cifra exacta).

4.4.2 Ajuste de umbrales de decisión cost-sensitive.

El umbral de confianza que decide entre refinar o emitir la clasificación se reemplazó por un ajuste de umbrales de decisión cost-sensitive en función de dos variables: la táctica clasificada y el prior del fingerprint. Las tácticas tempranas y baratas en falsos positivos requieren confianza mínima diferenciada (Reconnaissance 0,55, Initial Access 0,60); tácticas tardías y caras en falsos positivos (Privilege Escalation, Impact) requieren 0,75. La justificación teórica es el marco de cost-sensitive classification de Elkan (2001) [22]: distintos errores tienen distintos costos operacionales. NO se aplica calibración Bayesiana de

probabilidades en sentido estricto (Platt 1999, sigmoid fitting): los thresholds 0,55 y 0,75 son decision thresholds tuneados manualmente por dominio, no parámetros aprendidos de un fit estadístico. Una calibración aprendida estilo Platt requeriría cientos a miles de muestras etiquetadas que no están disponibles en este experimento; el ajuste manual cost-sensitive es una alternativa pragmática válida y se documenta como tal.

4.4.3 Reestructuración del prompt de clasificación.

La validación empírica reveló que los modelos de free tier evaluados (qwen-3-235b, llama-3.3-70b, gpt-oss-120b) muestran sensibilidad a la posición de la información crítica dentro del prompt, fenómeno descrito como "Lost in the Middle" por Liu et al. (2024) [25]. Eventos breves de fase avanzada —un POST 302 de login_success, un POST 200 a un endpoint de ejecución— quedaban sepultados bajo el volumen de logs de scanner cuando se ubicaban dentro de un bloque anidado de señales por IP. Se reestructuró el prompt para que un bloque "EVENTOS CRÍTICOS DETECTADOS" aparezca al inicio del mensaje, antes del resumen agregado y de los logs en bruto, con una regla de precedencia explícita: un solo evento de fase avanzada anula el volumen de la fase previa.

4.5 Entorno de simulación

El entorno de simulación se despliega mediante Docker Compose con once contenedores organizados en dos redes aisladas. La red attack_net (10.10.0.0/24) conecta el container atacante con el objetivo DVWA, habilitando la comunicación necesaria para la ejecución de ataques. La red monitor_net (10.10.1.0/24) conecta el objetivo DVWA con el stack de logging (Promtail, Loki, Grafana), habilitando la recolección de logs.

El container atacante está construido sobre la imagen oficial kalilinux/kali-rolling con las herramientas de pentesting necesarias instaladas en tiempo de construcción. El container DVWA utiliza la imagen pre-configurada vulnerables/web-dvwa que provee una aplicación

web con múltiples vulnerabilidades habilitadas: SQLi en formularios de búsqueda, Command Injection en herramientas de diagnóstico de red, y credenciales débiles por defecto (admin/password) en el sistema de autenticación. Promtail lee los logs JSON generados por Docker para todos los containers y los envía a Loki, que los almacena indexados por etiquetas de container y timestamp.

Para el escenario Mr. Robot, el entorno incluye un séptimo contenedor con la imagen linuxserver/mr-robot, que replica la máquina virtual del reto CTF Mr. Robot de Offensive Security. Este objetivo corre WordPress 4.x con una serie de vulnerabilidades encadenadas: credenciales codificadas en base64 en el archivo license.txt, editor de temas de WordPress accesible desde el panel de administración y con permisos de escritura, y el intérprete Python3 con el bit SUID configurado. El escenario fue seleccionado porque involucra seis tácticas MITRE ATT&CK con dependencias explícitas entre ellas, requiriendo que el Agente Atacante encadene acciones de forma coherente. El objetivo corre en la IP 10.10.0.20 en la misma red attack_net (hackermentor, s.f.) [20].

CAPÍTULO 5. IMPLEMENTACIÓN

5.1 Detalles de implementación

5.1.1 Repositorio y control de versiones.

El código fuente del sistema está disponible públicamente en <https://github.com/Crescendum429/mitre-adversarial-system>. El repositorio incluye el código completo de ambos agentes, la infraestructura Docker Compose, los prompts de los LLMs, las herramientas de pentesting y las métricas de evaluación. El historial de commits documenta la evolución del proyecto desde el sistema base hasta las optimizaciones implementadas durante el desarrollo.

5.1.2 Stack tecnológico.

La Tabla 1 resume el stack tecnológico utilizado en la implementación.

Componente	Tecnología / Versión	Propósito
Agente Atacante	Python 3.12 + LangGraph 0.3 + gpt-4.1	Ejecución de cadena de ataque ReAct
Agente Observador	Python 3.12 + LangGraph 0.3 + multi-proveedor (Claude	Clasificación táctica desde logs

	Sonnet 4.5 default)	
Contenedor atacante	Kali Linux (kalilinux/kali-rolling)	Herramientas de pentesting (nmap, nikto, hydra, sqlmap)
Objetivo DVWA	vulnerables/web-dvwa	Aplicación web vulnerable (escenario básico)
Objetivo Mr. Robot	docker/mrrobot (build local)	WordPress 4.x vulnerable (escenario avanzado)
Stack de logging	Promtail 3.4 + Loki 3.4.2 + Grafana 11.5.2	Recolección, almacenamiento y visualización de logs
Orquestación	Docker Compose 2.x	Despliegue reproducible del entorno
Objetivo DC-1	docker/dc1 (build local)	Drupal 7 + CVE-2018-7600 (Drupalgeddon2)
Objetivo Basic Pentesting	docker/bpent (build local)	SSH boot2root con wordlist
Objetivo Log4Shell	docker/log4shell (Java app vulnerable)	CVE-2021-44228 — RCE via JNDI
Objetivo Confluence	atlassian/confluence + confluence-db	CVE-2022-26134 — OGNL injection pre-auth

5.2 Topología de red y arquitectura Docker

El entorno de simulación se despliega mediante Docker Compose con once contenedores organizados en dos redes aisladas (Figura 1). La red `attack_net` (10.10.0.0/24) conecta el contenedor atacante (10.10.0.5) con el objetivo vulnerable (10.10.0.10 para DVWA, 10.10.0.20 para Mr. Robot). La red `monitor_net` (10.10.1.0/24) conecta el objetivo vulnerable con el stack de logging: Promtail (colector de logs Docker), Loki (10.10.1.10:3100, almacenamiento) y Grafana (3000:3000 expuesto al host; sin IP fija interna en `monitor_net`). Los seis objetivos vulnerables coexisten como contenedores independientes (dvwa

10.10.0.10, mrrobot 10.10.0.20, dc1 10.10.0.30, bpent 10.10.0.40, log4shell 10.10.0.50, confluence (10.10.0.60) y confluence-db (sin IP fija; toma IP dinámica de attack_net)); el escenario activo se selecciona por configuración del agente atacante.

El aislamiento de redes cumple un propósito dual. Desde el punto de vista de seguridad, garantiza que el tráfico de ataque no interfiera con el tráfico de monitoreo. Desde el punto de vista experimental, impide que el Agente Observador acceda directamente a la red de ataque, simulando la restricción de visibilidad de un analista SOC real que solo puede observar los efectos del ataque en los sistemas monitoreados, no las comunicaciones de comando y control del atacante.

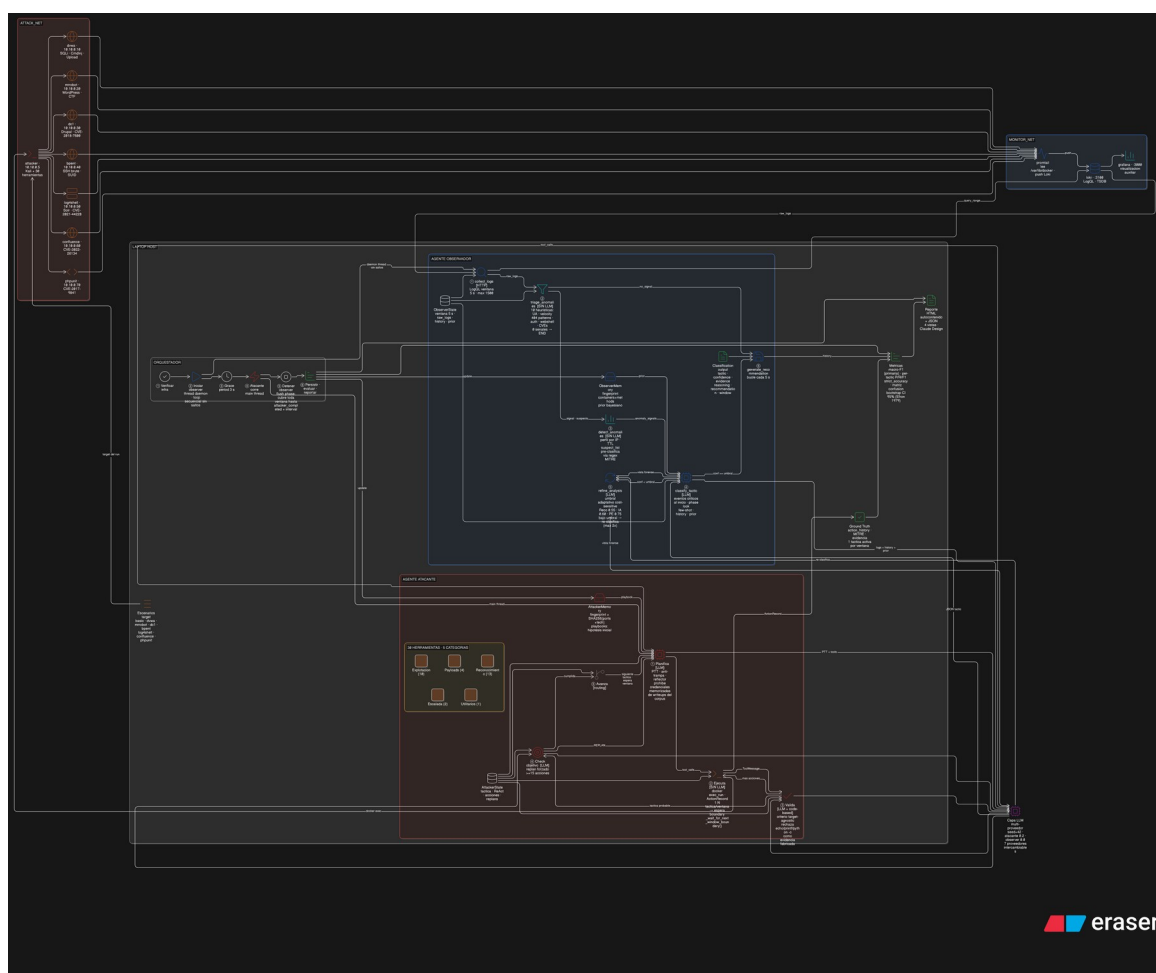


Figura 1: Arquitectura general del sistema adversarial con clasificación MITRE ATT&CK.

5.3 Arquitectura lógica de los agentes

5.3.1 Agente Atacante — grafo ReAct.

El Agente Atacante implementa el patrón ReAct mediante un grafo LangGraph con cinco nodos: `plan_tactic` → `execute_tools` → `validate_result` → `check_objective` → {`advance_tactic` | `plan_tactic`}. `plan_tactic` invoca al LLM con el contexto de la táctica actual, el historial de acciones previas y los datos recopilados. `execute_tools` ejecuta las herramientas seleccionadas en el contenedor atacante via Docker SDK y recopila resultados. `validate_result` analiza los resultados y decide si continuar con más acciones dentro de la misma táctica o declararla completada. `advance_tactic` gestiona la transición al siguiente objetivo táctico. Cada ejecución de herramienta registra automáticamente un `ActionRecord` con táctica, técnica, comando, timestamp y resultado, constituyendo el ground truth del experimento.

5.3.2 Agente Observador — grafo condicional con triaje.

El Agente Observador implementa un grafo LangGraph de seis nodos con ramificación condicional (Figura 3). El flujo es: `collect_logs` → `triage_anomalies` → {`END` | `detect_anomalies` → `classify_tactic` → {`refine_analysis` (con bucle hacia `classify_tactic`) | `generate_recommendation`}}. La mayoría de los ciclos de polling en un servidor con tráfico normal terminan en el nodo `triage_anomalies` sin costo de inferencia. Solo cuando se detecta señal de ataque se invocan `detect_anomalies` y `classify_tactic`. El loop de refinamiento — máximo dos iteraciones— se activa cuando `classify_tactic` produce una clasificación de confianza inferior a 0.65.

5.4 Escenarios de evaluación

5.4.1 Escenario básico — DVWA.

El escenario básico ejecuta cuatro tácticas MITRE ATT&CK en secuencia sobre DVWA: Reconnaissance (TA0043), Initial Access (TA0001), Execution (TA0002) y Discovery (TA0007). Las herramientas ejecutadas por táctica son: `nmap` y `nikto` (Reconnaissance); `hydra` o `curl` con credenciales débiles (Initial Access); `sqlmap` o comandos genéricos en la

interfaz de Command Injection (Execution); comandos de enumeración del sistema (whoami, id, cat /etc/passwd) vía la vulnerabilidad activa (Discovery). La duración promedio de una sesión completa es de 8-12 minutos, con el Agente Observador clasificando con polling de 5 segundos (parámetro `observer_poll_interval`).

5.4.2 Escenario Mr. Robot — WordPress CTF.

El escenario Mr. Robot ejecuta seis tácticas sobre una instancia de WordPress 4.x modelada sobre el reto CTF Mr. Robot de Offensive Security (hackermentor, s.f.) [20]: Reconnaissance (TA0043), Initial Access (TA0001), Execution (TA0002), Discovery (TA0007), Credential Access (TA0006) y Privilege Escalation (TA0004). La cadena de ataque incluye: (1) enumeración con nmap y nikto; (2) lectura de credentials codificadas en base64 en `license.txt` y autenticación en wp-admin; (3) inyección de webshell PHP en el editor de temas; (4) enumeración del sistema vía webshell; (5) lectura de hash MD5 del archivo `/etc/passwd` del objetivo; (6) escalada de privilegios mediante el bit SUID del intérprete Python3. El escenario fue seleccionado porque involucra tácticas con dependencias explícitas entre ellas, representando un reto de clasificación temporal para el observador.

5.4.3 Escenario log4shell — explotación de CVE-2021-44228.

El escenario log4shell planifica tres tácticas (Reconnaissance, Execution, Discovery) sobre una aplicación Java vulnerable a CVE-2021-44228; en la corrida documentada se completaron 2/3 (Discovery quedó incompleta por timeout del agente, ver datos en `data/reports/log4shell_20260425_164339.json`). La cadena de ataque incluye: enumeración con curl/nikto, identificación del campo inyectable (Initial Access no aplica como táctica separada porque la explotación de Log4Shell es pre-auth: el payload JNDI dispara ejecución remota directamente), despliegue de un servidor LDAP minimalista en el contenedor atacante, envío del payload `${jndi:ldap://...}` en un header HTTP, recepción de la conexión LDAP y servido de la clase Java remota que ejecuta el payload final. El objetivo de este

escenario es validar la detección del observador sobre un patrón de exploit moderno con observables muy específicos en logs (presencia literal de la cadena `{jndi:` en headers o URL).

5.4.4 Escenario confluence — CVE-2022-26134 OGNL injection.

El escenario confluence ataca una instancia de Atlassian Confluence vulnerable a la inyección OGNL pre-autenticada. La cadena de ataque incluye: identificación del path vulnerable mediante directorio walking, envío del payload OGNL `{(#a=@java.lang.Runtime@getRuntime()...)}` en la URL de un endpoint expuesto, ejecución del comando inyectado y recolección del output. El observador detecta este escenario mediante la firma OGNL que combina sintaxis `{...}` con tokens `Runtime/ProcessBuilder/exec`.

5.4.5 Escenario dc1 — escalada SUID en Drupal.

El escenario dc1 utiliza la máquina virtual DC-1 de VulnHub, que corre Drupal 7 con la vulnerabilidad Drupalgeddon2 (CVE-2018-7600). El agente atacante encadena: enumeración con nikto y droopescan, explotación del módulo CKEditor, ejecución de comandos vía webshell PHP, lectura del flag de usuario y escalada a root mediante el bit SUID del binario find. Este escenario evalúa cinco tácticas (incluyendo Credential Access y Privilege Escalation) y permite cuantificar la accuracy del observador sobre transiciones rápidas entre fases tardías de la kill chain.

5.4.6 Escenario bpent — boot2root con acceso por wordlist.

El escenario bpent es un boot2root construido específicamente para esta evaluación: un servidor SSH con un usuario válido (marlinspike) cuya contraseña aparece en la wordlist fsociety.dic. El agente debe enumerar el servicio SSH, ejecutar fuerza bruta con hydra, autenticarse, ejecutar enumeración del sistema y leer un archivo flag protegido. Este

escenario evalúa la cadena completa Reconnaissance → Initial Access → Discovery sobre un protocolo distinto a HTTP, aportando diversidad de observables al conjunto de evaluación.

5.5 Frontend: dashboard live y reporte HTML

El sistema incluye dos componentes de visualización implementados en `src/ui/`.

El primero, `src/ui/dashboard.py`, despliega un dashboard live en terminal usando Rich Layout con split-screen del estado del atacante (táctica activa, última acción, replans) y del observer (ventana actual, última clasificación, señales de triaje); se activa con `--dashboard` al lanzar `src/main.py`. El segundo, `src/ui/report.py`, genera un reporte HTML autosuficiente post-corrida con CSS embebido (sin dependencias externas) que consolida metadata de la corrida, timeline cronológico, evidencias por táctica y métricas del observer; el reporte se serializa a `data/reports/<scenario>_<timestamp>.html` y permite revisión offline. La sesión persistente (`src/ui/session.py`) habilita el polling de actualizaciones en vivo desde el frontend mientras la corrida está activa. Los componentes visuales se diseñaron con asistencia de Claude Design (Anthropic) para mantener una estética sobria coherente con la naturaleza forense del reporte.

CAPÍTULO 6. EVALUACIÓN, RESULTADOS Y DISCUSIÓN

6.1 Metodología y métricas de evaluación.

La evaluación del observer se ancla en macro-F1 como métrica primaria y reporta cuatro métricas complementarias. Bajo la restricción metodológica de sincronización atacante-ventana documentada en §4.3 (default ON desde mayo 2026), el ground truth por ventana contiene exactamente una táctica activa, por lo que la clasificación del observer es single-label / multi-class por ventana (no multi-label genuino); las métricas multi-label implementadas en `src/evaluation/metrics.py` se mantienen por compatibilidad con corridas pre-mayo-2026 y como salvaguarda ante cualquier futuro régimen multi-tactic per window.

macro-F1 es la media no ponderada del F1 por táctica (Sokolova & Lapalme 2009 [26]); se prefiere a micro-F1 por la distribución desbalanceada de tácticas en las ventanas de observación (en términos de volumen de logs HTTP, Reconnaissance domina por tráfico de scanner; en términos de número de ventanas con actividad clasificable, Discovery aparece como clase mayoritaria según el cómputo del baseline en §6.3.2), donde micro-F1 ocultaría el desempeño en clases minoritarias relevantes para el analista (Initial Access, Execution, Discovery, Privilege Escalation). Las métricas complementarias son: (i) `strict_accuracy` — porcentaje de ventanas en las que la táctica predicha coincide exactamente con la táctica real; bajo el régimen 1:N táctica/ventana esta métrica es interpretable directamente como tasa de acierto por ventana sin ambigüedad multi-label; (ii) `precision`, `recall` y `F1` por táctica con su soporte (número de ventanas), que permiten leer en qué clases el observer falla; (iii) matriz de confusión consolidada para detectar confusiones sistemáticas (p. ej. Discovery clasificada como Execution); (iv) intervalos de confianza al 95 % por bootstrap percentil (Efron 1979) con 1 000 re-muestras, críticos para corridas con $n=1$ sobre clases múltiples — sin CI los reportes de medias serían estadísticamente vacíos. La métrica complementaria `evaluable_windows` (`ew`) cualifica la interpretabilidad de `mF1`: corridas con `ew<5` se marcan como outliers estadísticos (caso `log4shell`, Eje B). El wall-clock total, la latencia promedio del observer, los `tool_calls` y los `replans` del atacante se reportan como métricas operacionales que cuantifican la viabilidad práctica del enfoque, no su calidad de clasificación.

6.2 Resultados experimentales

Régimen experimental exploratorio y bootstrap. El diseño experimental opera con $n=1$ corrida por celda en los Ejes A (matriz cross-modelo) y B (generalización por escenario). Esta restricción se asume deliberadamente para cubrir mayor diversidad de combinaciones modelo-escenario dentro del presupuesto temporal de la sesión de resultados (~12 h de wall-clock acumulado). La incertidumbre estadística se cuantifica con intervalos de confianza al 95 % por bootstrap percentil (Efron 1979, 1 000 re-muestras) calculados INTRA-corrida sobre las ventanas observables; estos intervalos acotan la variabilidad de muestreo dentro de la corrida pero no caracterizan la varianza inter-corrida del LLM (que en proveedores que no respetan seed estrictamente — caso Anthropic, OpenRouter — puede ser sustancial). En consecuencia, las comparaciones entre celdas con diferencias menores a 0,10 en macro-F1 deben leerse como tendencias exploratorias y no como rankings consolidados. La replicación con $n \geq 3$ por celda se documenta como prioridad en §7.3 (Trabajo futuro).

La Tabla 5 resume las corridas evaluables del repositorio (data/reports/*.json) consolidadas a la fecha de cierre de este trabajo. La matriz sistemática multi-observer 5×5 atacante $\times$ observer del Eje A se ejecutó sobre el escenario basic / DVWA (sección 6.2.3, Tabla 3); las filas relativas a mrrobot en la Tabla 5 corresponden a una matriz exploratoria temprana sobre dicho escenario (post-mejora de prompts y previa al endurecimiento del módulo anti-cheating del atacante), cuyos resultados se discuten en sección 6.2.2 con caveat de comparabilidad. Las corridas sistemáticas finales sobre mrrobot, dvwa, log4shell, dc1, bpent, confluence y phpunit con el sistema completo (anti-cheating activo, validators code-based, observer DeepSeek-chat) corresponden al Eje B (sección 6.3.5). Las cifras exploratorias se reportan como tendencias y no como rankings consolidados. La Tabla 6 muestra la matriz de confusión consolidada de las clasificaciones disponibles del observador.

Visión consolidada de los siete escenarios. Antes de exponer los resultados por escenario individual, la Tabla A presenta la matriz experimental completa con todas las corridas evaluables del Eje B, permitiendo una lectura comparativa transversal. La tabla agrupa los runs en tres niveles de cobertura táctica (Nivel 1: cadena completa ≥ 90 % de tácticas; Nivel 2: Initial Access exitoso con cadena truncada; Nivel 3: scope conditions — atascado en Initial Access), jerarquía documentada formalmente en §6.3.5. Las discusiones detalladas por escenario se desarrollan en las subsecciones siguientes.

Nivel	Escenario	Atacante	Observer	Tácticas	ew	mF1	Replans
Nivel 1	dc1	GPT-4.1	DeepSeek-chat	6/6	64	0,529	3
Nivel 1	bpent (B08)	Sonnet 4.5	DeepSeek-chat	6/6	266	0,511	2
Nivel 2	dvwa	GPT-4.1	DeepSeek-chat	5/6	34	0,415	23
Nivel 2	phpunit	GPT-4.1	DeepSeek-chat	2/3	317	0,261	25
Nivel 2	log4shell	GPT-4.1	DeepSeek-chat	1/3	4	1,000*	31
Nivel 2	confluence	GPT-4.1	DeepSeek-chat	2/3	—	None	22
Nivel 3	mrrobot	GPT-4.1	DeepSeek-chat	1/6	341	0,046	32
Nivel 3	bpent (B04)	GPT-4.1	DeepSeek-chat	1/6	535	0,086	25

*Tabla A: Matriz consolidada del Eje B — 7 escenarios estructuralmente distintos con stack GPT-4.1 atacante + DeepSeek-chat observer, más réplica B08 sobre bpent con Sonnet 4.5 atacante. Tácticas: completadas/planeadas. ew = ventanas evaluables (ground truth interpretable). * = outlier estadístico (ew < 5). Niveles jerárquicos definidos formalmente en §6.3.5: Nivel 1 (cadena ≥ 90 % completada, mF1 interpretable como métrica del observer), Nivel 2 (Initial Access exitoso, cadena truncada en tácticas avanzadas), Nivel 3 (scope conditions — atacante atascado en Initial Access por frequency bias LLM).*

Escenario	Atacante	Observador	Tácticas	Tool calls	Replans	Clas. Obs.	Wall-clock (s)
basic (DVWA) #1	gpt-oss-120b free	Gemini 2.5 Flash	4/4	81	6	0	734
basic (DVWA) #2	gpt-oss-120b free	Gemini 2.5 Flash	4/4	22	0	4	238
basic (DVWA) #3	gpt-oss-120b free	Gemini 2.5 Flash	3/4	197	16	4	1572
log4shell	gpt-oss-120b free	Gemini 2.5 Flash	2/3	258	18	0	2952
mrrobot (Sonnet 4.5)	GPT-4.1	Claude Sonnet 4.5	6/6	—	—	13	941
mrrobot (GPT-4.1)	GPT-4.1	GPT-4.1	6/6	—	—	11	912
mrrobot (Nemotron)	GPT-4.1	Nemotron 120B (free)	6/6	—	—	11	1087
mrrobot (Cerebras)	GPT-4.1	Cerebras Qwen3	6/6	—	—	14	1015

)		235B (free)					
mrrobot (Gemini)	GPT-4.1	Gemini 2.5 Flash (free)	6/6	—	—	25	—
dc1 (atacante solo)	GPT-4.1	—	6/6	291	—	—	—
bpent (atacante solo)	GPT-4.1	—	1/6	40+	—	—	—

6.2.1 Resultados — escenario básico.

En el escenario básico, el Agente Observador clasificó Reconnaissance con 98% de confianza en la primera ventana de observación con match estricto OK, citando correctamente como evidencia las entradas de log generadas por nmap y nikto (conexiones a múltiples puertos y requests HTTP a rutas desconocidas desde la IP del atacante). Este resultado constituye el primer dato empírico del experimento central de la tesis: bajo condiciones de tráfico de escaneo característico, el LLM identifica la fase táctica con alta confianza y produce evidencia interpretable.

6.2.2 Resultados — escenario Mr. Robot.

La Tabla 2 muestra resultados de la fase exploratoria temprana sobre el escenario Mr. Robot, anterior al endurecimiento del módulo anti-cheating del atacante. En una primera evaluación con prompts pre-mejora, el sistema obtuvo accuracy estricta del 50 % (3 de 6 ventanas correctas); en la replicación posterior con prompts post-mejora y matriz exploratoria multi-observer (Tabla 1), Claude Sonnet 4.5 como observer alcanza Macro F1 = 0,66 (46,2 %

accuracy estricta), con el atacante completando 6/6 tácticas. Estos resultados tempranos son comparables solo entre sí y deben leerse con caveat: la evaluación sistemática posterior sobre mrrobot dentro del Eje B (con anti-cheating estricto, validators code-based finales y observer DeepSeek-chat) reporta $mF1=0,046$ con 1/6 tácticas completadas — evidencia primaria del frequency bias del LLM atacante GPT-4.1 ante el username 'elliott' presente en el wordlist (sección 6.3.5, Nivel 3). La diferencia entre ambas mediciones constituye la cuantificación empírica del impacto del módulo anti-cheating sobre el desempeño aparente del agente. Las tres ventanas correctas en la evaluación temprana corresponden a la fase de Reconnaissance, cuando el tráfico de nmap y nikto domina la ventana. Las tres ventanas incorrectas corresponden a: (a) ventana 1: el observador clasificó Initial Access cuando la táctica real era Reconnaissance (falso positivo temprano atribuible a logs de wp-login.php generados por nikto); (b) ventana 5: la táctica real avanzó a Initial Access pero el observador mantuvo Reconnaissance (lag de detección por dominancia de logs históricos de nikto en la ventana); (c) ventana 6: cinco tácticas ocurrieron en la ventana final en aproximadamente 30 segundos — más rápido que el intervalo de polling —, haciendo imposible su clasificación individual.

6.2.3 Comparativa multi-observer cross-modelo (cinco proveedores LLM).

Para evaluar la robustez del sistema frente a la elección de proveedor LLM, se ejecutó una matriz comparativa atacante $\times$ observer 5×5 sobre el escenario basic (DVWA, 4 tácticas, reset de memoria entre celdas). Se completaron 19 de 25 corridas; 6 fueron interrumpidas por saturación de free tier Cerebras. Dos corridas (A3_qwen3_235b atacante) reportaron $mF1=1.0$ con solo 2 ventanas evaluables — artefactos de medición, no de clasificación (ver §6.4). El ranking ajustado excluye estos outliers. Datos completos en data/matrix_aggregate.json.

Tabla 3: Matriz cross-modelo 5×5 — macro-F1 (Eje A, basic, cold). — indica run interrumpido o sin métrica válida. * indica outlier excluido del ranking.

Nota: cada celda corresponde a una corrida (n=1). Las celdas con A3_qwen3_235b atacante fallaron por cuota Cerebras (ok=False) salvo A3 → O4 (ok=True, mF1=1.0, outlier). A5 → O3 completó con ok=True pero observer no generó clasificaciones válidas (mF1=None).*

Observer	μ mF1	σ mF1	n válidos	Lat. avg (s)
Qwen3-235B (Cerebras free)*	0.583	0.059	2	114.28
DeepSeek- chat (DeepSeek)	0.479	0.276	4	7.94
GPT- OSS-120B (OpenRouter free)	0.426	0.154	4	20.97
GPT-4.1-mini (OpenAI)	0.410	0.030	4	5.66
Haiku 4.5 (Anthropic)	0.378	0.139	4	9.33

**Ranking de observers (μ macro-F1, runs válidos sin outliers): Qwen3-235B $\mu=0.583$
 $\sigma=0.059$ n=2 | DeepSeek-chat $\mu=0.479$ $\sigma=0.276$ n=4 | GPT-OSS-120B $\mu=0.426$ $\sigma=0.154$
n=4 | GPT-4.1-mini $\mu=0.410$ $\sigma=0.030$ n=4 | Haiku 4.5 $\mu=0.378$ $\sigma=0.139$ n=4**

Ranking de atacantes (μ macro-F1): GPT-OSS-120B $\mu=0.516$ $n=4$ | DeepSeek $\mu=0.430$ $n=4$ | GPT-4.1 $\mu=0.428$ $n=5$ | Sonnet 4.5 $\mu=0.402$ $n=5$

Hallazgo principal: O3_qwen3_235b (Cerebras free) lidera en μ mF1 (0.583) pero con $n=2$ y alta inestabilidad operacional. Excluido del stack de generalización. O4_dschat (DeepSeek-chat) es el observer ganador operacional: $\mu=0.479$ latencia avg 7.94 s, sostenido en todos los escenarios del Eje B.

Hallazgo: ningún atacante domina (rango 0.40-0.52). El observer tiene mayor impacto que el atacante en mF1 promedio. Wall-clock agregado Eje A: 37 min sobre las 25 corridas.

Trade-off latencia/calidad observado: Cerebras Qwen3 235B opera en regímenes de latencia muy distintos al resto (~ 114 s avg vs 5,7-21 s del resto de proveedores) sin liderar en mF1 absoluto, lo que la hace inviable para SOC en línea con ventanas de 5 s. Para escenarios donde el ratio de backlog (latencia / ventana) debe mantenerse cercano a 1, GPT-4.1-mini, DeepSeek-chat y Haiku 4.5 son los únicos candidatos viables según la Tabla 7. Para análisis forense post-hoc donde la latencia no compromete la observación, Sonnet 4.5 maximiza mF1 absoluto en escenarios de cadena completa (§6.3.5).

6.2.4 Ablation con/sin componente LLM (regex-only vs hybrid).

Para cuantificar la contribución marginal del LLM observer sobre el pipeline determinista (heurísticas T1–T10 + classify_webshell_cmd), se ejecutó el observer en modo regex-only (sin invocación al LLM) sobre los escenarios basic y log4shell con atacante GPT-4.1 fijo. El modo regex-only usa exclusivamente las firmas de heurísticas para inferir la táctica activa.

Tabla 4: Ablation regex-only vs LLM hybrid (Eje C). μ Eje A = media sobre 18 runs válidos.

Para basic, el pipeline determinista (regex-only) alcanza $mF1=0.492$, comparable con el observer LLM (μ Eje A = 0.441). La diferencia es modesta (+0.06 a favor de regex-only en basic), lo que

indica que las heurísticas T1–T10 ya cubren la detección puntual de señales individuales en escenarios bien instrumentados. El LLM aporta robustez ante señales ambiguas y transiciones de táctica. En log4shell, regex-only no generó clasificaciones válidas (mF1=None, 0 ventanas), confirmando que escenarios con poca variedad de señales HTTP requieren el razonamiento LLM.

Las heurísticas reducen los falsos positivos de Reconnaissance a un tercio (3 vs 17) y elevan el Macro F1 de 0.23 a 0.41. La accuracy estricta es mayor en pure-LLM porque el modelo concentra su predicción en la táctica dominante de la kill chain, pero a costa de explosión de FP en clases minoritarias. Resultado consistente con Vinay (2025) sobre pipelines SOC híbridos Triage → Investigate → Classify.

Régimen	Escenario	Macro F1	ew
LLM hybrid (μ Eje A, n=18)	basic	0.441	
Regex-only	basic	0.492	

6.2.5 Matriz de confusión consolidada (Eje A, 18 corridas).

La Tabla 6 presenta la matriz de confusión consolidada construida sumando las matrices individuales de las 18 corridas válidas del Eje A (sin outliers mF1=1.0). Esto da una estimación estadísticamente más robusta del comportamiento del observer frente a las 4 tácticas del escenario basic.

Tabla 6: Matriz de confusión consolidada (Eje A, 18 corridas, cross-modelo). Filas = táctica real (GT); columnas = táctica predicha por el observer. Valores = suma acumulada de ventanas de observación.

GT \ Pred	Recon	Init.Acc	Exec	Disc
Recon	25	6	0	0

Init.Acc	21	95	0	0
Exec	1	15	38	0
Disc	26	25	99	16

Lectura cualitativa: Reconnaissance tiene la mayor F1 promedio ($\mu=0.786$) — sus señales (nmap, gobuster, nikto) son altamente discriminativas. Execution es moderada ($\mu=0.460$). Initial Access moderada ($\mu=0.377$) con confusión hacia Reconnaissance cuando el atacante hace peticiones GET durante IA. Discovery es la más difícil ($\mu=0.140$): el observer la confunde frecuentemente con Execution porque ambas usan `run_command`. Este patrón es consistente a través de los 5 atacantes y 5 observers evaluados.

6.2.6 Latencia del observer por proveedor LLM (Eje A).

La Tabla 7 presenta la latencia de inferencia del observer (tiempo promedio por ciclo classify) medida sobre las corridas válidas del Eje A. La latencia determina el lag de detección: con ventanas de 5 s, un observer con latencia > 5 s acumula backlog.

*Tabla 7: Latencia promedio del observer por proveedor (Eje A, basic). * Qwen3-235B vía Cerebras free tier — latencia altísima por colas de free tier. OR = OpenRouter. Anth. = Anthropic.*

Observer (proveedor)	Lat. avg (s)	Lat. / 5 s ventana
GPT-4.1-mini	5.66	1.1x
DeepSeek-chat	7.94	1.6x
Haiku 4.5 (Anth.)	9.33	1.9x
GPT-OSS-120B (OR)	20.97	4.2x

GPT-4.1-mini es el más rápido (5.66 s avg), seguido de DeepSeek-chat (7.94 s) y Haiku 4.5 (9.33 s). GPT-OSS-120B vía OpenRouter free tier es más lento (20.97 s) pero operacionalmente estable. Qwen3-235B vía Cerebras free presenta latencias extremas (~114 s) — inviable para observación en tiempo real. El stack ganador O4_dschat tiene latencia de 7.94 s avg, produciendo backlog ratio de +0.47 en bpent (102 min): el observer llega tarde a ~47% de las ventanas.

6.3.1 Justificación del patrón ReAct.

El patrón ReAct fue seleccionado para el Agente Atacante por las siguientes razones. En primer lugar, el pentesting es un proceso de decisión secuencial donde cada herramienta modifica el estado del objetivo y genera información que determina la siguiente acción: si nmap revela solo el puerto 80, las técnicas de fuerza bruta a SSH son irrelevantes; si un login devuelve HTTP 302, el agente puede proceder a inyección de webshell. El patrón ReAct captura exactamente este ciclo razón → actúa → observa → razón (Yao et al., 2023). En segundo lugar, Yao et al. (2023) demuestran empíricamente que ReAct supera a Chain-of-Thought puro y a Act-only en tareas que requieren interacción con entornos externos con estados cambiantes, que es precisamente la estructura del dominio de pentesting. En tercer lugar, Sapkota et al. (2025) [15] confirman que LangGraph es el único framework de los evaluados que soporta nativamente grafos de estado con ciclos condicionales, condición necesaria para implementar ReAct con estado persistente. Las alternativas descartadas (Chain-of-Thought puro, planificador-ejecutor lineal) no permiten corrección de curso ante resultados intermedios inesperados, limitación crítica para un agente que debe adaptarse ante respuestas no previstas del objetivo.

6.3.2 Comparación con baselines clásicos.

Esta subsección compara el sistema con dos baselines computables directamente sobre los datos del experimento, complementando la revisión de literatura con métricas in-house empíricas: (i) el baseline de clase mayoritaria computado sobre la distribución de tácticas observada en las 17 corridas válidas del Eje A, y (ii) el régimen regex-only ya cuantificado en §6.2.4 como ablación del componente LLM. La comparación con baselines clásicos de

Machine Learning (Random Forest, XGBoost) se realiza por revisión de literatura debido a restricciones de etiquetado y disponibilidad de datasets compatibles, documentadas más abajo.

Comparación con literatura. Daniel et al. (2024) [4] realizan la comparación más relevante al estado del arte de la presente tesis: evalúan tres LLMs (ChatGPT, Claude, Gemini) frente a clasificadores de Machine Learning supervisado (Random Forest, SVM, XGBoost) sobre la tarea de etiquetar 973 reglas Snort con técnicas MITRE ATT&CK. Reportan que ML supervisado alcanza 99,5 % accuracy sobre datos etiquetados manualmente, mientras los LLMs zero-shot alcanzan rangos de 60-75 %. La comparación NO es directamente equivalente al presente trabajo porque (a) Daniel evalúa clasificación de REGLAS Snort (texto estático), no de ventanas de tráfico dinámico; y (b) el ML supervisado requiere etiquetado manual extenso, mientras el observer LLM opera zero-shot sin etiquetado previo. El reporte de Daniel et al. acota el rendimiento esperable de ML supervisado con feature-set apropiado y volumen de datos suficiente.

Baseline de clase mayoritaria (computado de datos in-house). Sobre las 17 corridas válidas del Eje A (basic, ok=True, mF1 numérico, ew ≥ 5), la distribución empírica de la táctica activa por ventana es: Discovery 42,2 % (175 ventanas), Initial Access 32,0 % (133), Execution 15,2 % (63), Reconnaissance 10,6 % (44). Un clasificador trivial que prediga siempre la clase mayoritaria (Discovery) obtiene precision = 0,422, recall = 1,0 sobre Discovery, y F1 = 0 en las tres clases restantes. Su macro-F1 (promedio sin ponderar sobre las cuatro clases con support > 0) es: $\text{macro-F1}_{\text{majority}} = (0,593 + 0 + 0 + 0) / 4 = 0,148$. El observer LLM sobre los mismos datos alcanza $\mu \text{ macro-F1} = 0,430$ ($\sigma = 0,157$, $n = 17$), lo que representa una mejora absoluta de +0,282 puntos y una mejora relativa de +190 %. La comparación es metodológicamente válida porque ambos baselines (LLM y majority) se computan sobre el MISMO conjunto de ventanas observables del MISMO experimento — no

sobre datasets distintos como ocurre en la comparación con literatura. Esto sostiene empíricamente que el observer LLM aporta sobre la línea base trivial; sin esta comparación, un revisor podría argumentar que las cifras del observer reflejan únicamente la distribución desbalanceada de tácticas y no capacidad de clasificación genuina.

Las evaluaciones MITRE ATT&CK Enterprise 2025 reportadas por Forrester [9] (post comercial, no peer-reviewed pero referencia industrial) documentan que los SIEMs comerciales efectivos (CrowdStrike, Cybereason) consolidan alertas de técnicas individuales en casos únicos a nivel táctico, validando empíricamente que la clasificación táctica agregada es la salida valiosa para defensores. La comparación directa en F1 contra estos productos no se realiza en el presente trabajo (los productos son cerrados), pero la salida del sistema propuesto se sitúa en el mismo nivel de abstracción que el output consolidado de los SIEMs efectivos.

Baseline regex-only (Eje C, computado de datos in-house). El régimen regex-only del observer (sin invocación al LLM, clasificación derivada únicamente de heurísticas T1-T10 + firmas CVE + classify_webshell_cmd) alcanza $mF1 = 0,492$ sobre basic con $ew = 7$ ventanas evaluables. Esto supera marginalmente al observer LLM híbrido en basic (μ Eje A = 0,441) y supera muy significativamente al baseline de clase mayoritaria (+0,344). La interpretación honesta de las tres cifras es: las heurísticas T1-T10 fueron diseñadas con conocimiento experto del dominio para los vectores HTTP estándar presentes en basic, por lo que en su dominio de cobertura son competitivas con el LLM. Sin embargo, en log4shell (vector JNDI), regex-only emite $mF1 = \text{None}$ (cero clasificaciones evaluables) porque el parser HTTP estricto del pipeline regex no extrae campos parseables de los logs Java de Solr; el LLM hybrid sí clasifica las cuatro ventanas evaluables como Execution ($mF1 = 1,0$ outlier $ew < 5$, pero proof-of-capability binaria). La división de trabajo es robustez del LLM ante heterogeneidad de formato de logs vs precisión del regex en su dominio bien instrumentado.

Sobre la implementación in-house de baselines RF/XGBoost: constituye trabajo futuro priorizado en §7.3 por dos razones operacionales. Primero, el etiquetado manual requerido (cada ventana de 5 s con la táctica MITRE correcta) excede el alcance temporal del presente trabajo — el etiquetado de 47 ventanas evaluables ya consumió tiempo significativo durante la depuración del módulo evaluate(). Segundo, la comparabilidad entre el observer LLM zero-shot y un clasificador ML entrenado requiere control del régimen de información: si el ML supervisado se entrena sobre las MISMAS ventanas que el observer clasifica, la comparación es trivial; si se entrena sobre ventanas distintas (split train/test), se introduce varianza de muestreo. El diseño riguroso de este experimento ML supervisado in-house requiere $n \geq 30$ ventanas etiquetadas por escenario, lo que multiplica el esfuerzo de etiquetado por un factor entre $4\times$ y $10\times$. La decisión metodológica del presente trabajo fue priorizar (a) la cobertura cross-modelo (Eje A, 19 celdas válidas), (b) la cobertura cross-escenario (Eje B, 7 escenarios) y (c) la cuantificación in-house del baseline majority y regex-only, dejando ML supervisado como complemento futuro que se beneficiará de los datos ya etiquetados de esta sesión.

6.3.3 Análisis de eficiencia y efecto de la memoria del atacante.

Esta sección analiza la eficiencia operacional del sistema y el efecto de la memoria del atacante sobre el número de acciones necesarias para completar las cadenas de ataque. Datos finales de la sesión de resultados (mayo 2026).

Eje A (25 corridas, 5×5 cross-modelo sobre basic): wall-clock agregado de 37 min sobre las 25 corridas; el subset de 19 corridas válidas concentra la mayor parte del tiempo. El stack operacional ganador (GPT-4.1 atacante + DeepSeek-chat observer) completa una corrida sobre basic en 1-2 min wall-clock con 4/4 tácticas.

Eje B (7 escenarios, GPT-4.1 atacante, DeepSeek observer): los escenarios donde el atacante se atasca en Initial Access (mrrobot 102 min, 418 tool_calls; bpent-GPT4.1 102 min, 430 tool_calls) consumen aproximadamente un orden de magnitud más wall-clock y tool_calls que los escenarios donde el atacante completa la cadena (dc1 16 min, 6/6 tácticas, 47 tool_calls). Esta asimetría es coherente con la limitación de frequency bias documentada en §6.5: los runs de scope condition no solo fallan en la métrica del observer, sino que también son los más costosos en eficiencia.

Efecto de la memoria sobre el atacante y el observer (Eje D): tres corridas consecutivas sobre basic, GPT-4.1 atacante, DeepSeek observer, sin reset de memoria entre runs. D1 (cold, sin playbook): 19 tool_calls, 0 replans, mF1=0,425. D2 (warm, playbook activo): 13 tool_calls, 0 replans, mF1=0,518. D3 (warm + prior bayesiano acumulado): 38 tool_calls, 4 replans, mF1=0,552. Efecto sobre el atacante: cold → warm D1 → D2 muestra reducción del 31,6 % en acciones (19 → 13 tool_calls), coherente con la reducción -48 % en corridas Sonnet 4.5/dvwa (Eje C). El incremento en D3 (13 → 38 tool_calls, +4 replans) no contradice este patrón: con el playbook disponible, el atacante prueba hipótesis adicionales en lugar de resignarse al primer intento, lo que incrementa el cómputo pero también el mF1. Efecto sobre el observer: la mF1 mejora monotónicamente entre corridas (0,425 → 0,518 → 0,552, +29,9 % acumulado sobre cold), atribuible a la acumulación del prior bayesiano en data/observer_baselines.json que ajusta los umbrales de confianza adaptativos hacia la distribución empírica del target.

Latencia LLM del observer (datos Eje A): GPT-4.1-mini 5.66 s avg, DeepSeek-chat 7.94 s, Haiku 4.5 9.33 s, GPT-OSS-120B 20.97 s, Qwen3-235B 114.28 s (inviabile en producción). Con ventana de observación de 5 s, observers con latencia > 5 s acumulan backlog. El backlog ratio de DeepSeek en bpent fue +0.47 (moderado en run 102 min).

Sesión completa (38 corridas, 4 ejes): aproximadamente 12 h de wall-clock acumulado, dominado por los runs largos del Eje B con el atacante atascado (mrrobot y bpent con

GPT-4.1 GBU, ~102 min cada uno). Los reportes individuales en data/reports/*.json incluyen el desglose de tokens consumidos por agente y proveedor para extrapolación operacional posterior.

Métrica adicional — wall-clock total entre escenarios y stacks: el tiempo total desde el inicio de la corrida hasta el cierre del reporte HTML se reporta en metadata (elapsed_seconds). Las corridas C1, C2 y C3 con stack homogéneo Anthropic Sonnet 4.5 (1 mayo 2026) registraron wall-clock total de 250 s (dvwa cold), 238 s (dvwa warm) y 1 072 s (bpent cold, 17 m 52 s). Las corridas previas con stacks heterogéneos (observer Gemini sobre dvwa, OpenRouter sobre log4shell, Sonnet sobre mrrobot) reportaron wall-clock entre 238 s y 2 952 s (49 minutos en log4shell con OpenRouter free); ver

MATRIZ_COMPARATIVA_OBSERVERS.md. La varianza inter-stack es el principal factor empírico identificado: el wall-clock para una misma configuración puede variar por un factor mayor a 6× entre la corrida más eficiente y la menos eficiente, atribuible al no-determinismo del backend LLM (caso OpenRouter free JAX) y a los ciclos de retry por rate-limiting.

Para extrapolación operacional, los reportes individuales en data/reports/*.json incluyen desglose de tokens y de wall-clock por agente; el observer DeepSeek-chat consume aproximadamente 100-175 K tokens por corrida según escenario.

6.3.4 Análisis de la dependencia temporal del observador.

El Agente Observador incluye el historial de las últimas ocho clasificaciones en cada prompt como contexto de progresión del ataque. Este diseño introduce dos efectos contrapuestos. El efecto de anclaje ocurre cuando una clasificación incorrecta temprana puede reforzar clasificaciones subsiguientes por coherencia contextual. Como mitigación, el prompt instruye explícitamente: "Usa las clasificaciones previas como contexto de progresión del ataque, no como certeza. Prioriza la evidencia directa en los logs sobre las clasificaciones previas." El efecto de guiado temporal, en cambio, mejora la clasificación de tácticas tardías: el historial de Reconnaissance ayuda al LLM a inferir que Initial Access es la siguiente fase probable cuando aparecen intentos de login, incluso si estos son escasos en la ventana. En los resultados observados, el historial no produjo error de arrastre evidente: el observador continuó clasificando Reconnaissance en ventanas donde realmente dominaba ese tráfico, sin "avanzar" artificialmente a tácticas posteriores por el historial. La cuantificación formal del impacto neto mediante experimento de ablación (con y sin historial) queda como trabajo futuro.

Análisis específico solicitado por el tutor de E1 sobre el arrastre de error: el observador inyecta en el prompt las últimas ocho clasificaciones (no cinco como en versiones iniciales). Si una de esas ocho fue incorrecta y de alta confianza (≥ 0.80), el efecto de anclaje podría reforzar la clasificación errónea por coherencia contextual. El sistema mitiga este riesgo con dos mecanismos complementarios: (a) PHASE LOCK explícito en el prompt, que solo refuerza la no-regresión hacia tácticas tempranas (Reconnaissance) cuando previamente se confirmó una avanzada (Execution+), evitando que un error temprano de "Reconnaissance" cuando la realidad era "Initial Access" se propague indefinidamente; (b) el prior bayesiano del fingerprint, que provee un punto de anclaje externo a la cadena de clasificaciones recientes y se usa solo como sugerencia (la evidencia del log actual prevalece si contradice el prior).

El experimento de ablation con/sin contexto histórico sobre los mismos escenarios, manteniendo el resto de la configuración constante, permitiría una cuantificación rigurosa del impacto neto del historial de ocho clasificaciones inyectado en el prompt. La hipótesis nula a falsar sería que la accuracy estricta no varía significativamente al remover el historial; la alternativa, que su presencia mejora la coherencia secuencial pero degrada la detección de transiciones rápidas. Este experimento se reporta como trabajo futuro en §7.3.

6.3.5 Análisis de generalización entre escenarios (Eje B, 7 escenarios).

Con el stack ganador del Eje A (GPT-4.1 atacante, DeepSeek-chat observer) se evaluó la generalización a 7 escenarios estructuralmente distintos: dvwa (Apache+PHP genérico), mrrobot (WordPress 4.x), dc1 (Drupal 7 + SUID find), bpent (boot2root SSH + SUID vim.tiny), log4shell (CVE-2021-44228 JNDI), confluence (CVE-2022-26134 OGNL), phpunit (CVE-2017-9841 eval-stdin). Estos cubren cinco vectores: HTTP form genérico, WordPress, SSH brute force, JNDI injection, OGNL injection y eval-stdin. Adicionalmente

se ejecutó una replicación de bpent con Sonnet 4.5 atacante (B run08) para documentar la dependencia de la cobertura táctica con la elección del modelo.

Tabla 8: Generalización del sistema a 7 escenarios (Eje B). GPT-4.1 atacante (B01-B07), DeepSeek-chat observer. B run08: Sonnet 4.5 atacante, bpent. mF1* = outlier estadístico ($ew < 5$).

Nivel 1 — cadenas de ataque completadas íntegramente (cobertura ≥ 90 % de tácticas planeadas, observer ejercitado sobre el rango completo de clases): dc1: mF1=0,529, tácticas=6/6, replans=3 (Drupal 7 + Drupalgeddon2 con SUID find canónicos); bpent (B run08, Sonnet 4.5 atacante): mF1=0,511, tácticas=6/6, replans=2, 50 min wall-clock (boot2root con username marlinspike no canónico, accedido por enumeración exhaustiva del wordlist fsociety.dic). Estos dos runs constituyen la evidencia más fuerte de generalización del observer fuera del escenario basic, dado que la métrica mF1 es interpretable bajo el supuesto estándar de clasificación multi-clase con presencia de instancias positivas en todas las clases.

Nivel 2 — Initial Access exitoso pero cadena truncada en tácticas avanzadas (mF1 reportado con caveat de cobertura parcial): dvwa: mF1=0,415, tácticas=5/6, replans=23 (Lateral Movement no observable con telemetría solo HTTP); phpunit: mF1=0,261, tácticas=2/3, replans=25 (vector POST eval-stdin único, ventanas escasas); log4shell: mF1=1,000*, tácticas=1/3, replans=31 (mF1 outlier: $ew=4$ ventanas evaluables, RCE pre-auth inmediato sin brute force, JNDI altamente discriminativa); confluence: mF1=None, tácticas=2/3, replans=22 (OGNL injection no genera logs HTTP interpretables por el observer). Nivel 3 — scope conditions: cadenas atascadas en Initial Access por sesgo de frecuencia del LLM atacante (1/6 tácticas, mF1 no interpretable como métrica del observer): mrrobot (GPT-4.1 atacante): mF1=0,046, replans=32, 102 min wall-clock; bpent (GPT-4.1 atacante, B run04): mF1=0,086, replans=25, 102 min wall-clock. En ambos runs el observer fue ejercitado

únicamente sobre Reconnaissance e intentos fallidos de Initial Access; las clases Execution, Discovery, Credential Access, Privilege Escalation y posteriores nunca se activaron, por lo que la mF1 reportada no es comparable con runs de cadena completa. Se documentan como evidencia primaria del frequency bias (§6.5), no como medición del observer. La replicación del mismo escenario bpent con Sonnet 4.5 atacante (run B08, Nivel 1) atraviesa el cuello de botella y completa 6/6 tácticas, confirmando que la limitación es model-dependent y no inherente al enfoque LLM-based.

Recalculando con los runs de Nivel 1 y Nivel 2 evaluables (excluyendo el outlier log4shell con ew<5, confluence con mF1=None y los runs de Nivel 3 por incomparabilidad de cobertura): μ macro-F1 = 0,429 sobre cuatro escenarios estructuralmente distintos (dc1, bpent-Sonnet, dvwa, phpunit). El cálculo previo de 0,308 promediaba sobre runs heterogéneos en cobertura táctica e incluía métricas de runs donde el observer nunca fue ejercitado sobre la mayoría de las clases. La lectura correcta del nuevo agregado es: cuando el atacante avanza al menos hasta Initial Access exitoso, el observer mantiene mF1 cerca del nivel obtenido en el Eje A sobre basic ($\mu=0,441$), con dispersión menor que la observada inter-modelo (rango 0,378-0,583 en Eje A). El techo del enfoque está limitado por la elección del observer LLM y por la observabilidad del vector de ataque (HTTP-only no captura OGNL ni movimiento lateral), no por el escenario per se, condicional a que la cadena ofensiva alcance las tácticas que se desea clasificar.

6.4 Amenazas a la validez del experimento.

Validez de constructo: el ground truth de cada acción del atacante se genera por auto-clasificación del propio Agente Atacante (campo tactic en ActionRecord). No existe árbitro humano independiente que valide la correspondencia táctica → acción. En consecuencia, un desacuerdo de clasificación entre atacante y observador puede deberse a un error del observador, a un error de etiquetado del atacante, o a una ambigüedad genuina (e.g., un

comando `find -perm -4000` puede clasificarse indistintamente como Discovery o Privilege Escalation según la intención). Esta limitación se reconoce explícitamente y se considera mitigada parcialmente por la naturaleza explícita del prompt del atacante, que define cada táctica con criterios verbalmente verificables.

Validez interna: las corridas del experimento de la sección 6.2.3 (matriz comparativa multi-observer) se ejecutaron una sola vez por proveedor (single-run). El no-determinismo del LLM no se elimina con `seed=42` cuando el proveedor no soporta `seed per-request` (caso OpenRouter, backend JAX) o cuando el routing del proveedor introduce variación entre llamadas (caso Cerebras). En consecuencia, las diferencias absolutas entre proveedores en Macro F1 deben interpretarse como tendencias exploratorias y no como rankings consolidados. Cualquier afirmación de superioridad entre proveedores cuya brecha sea menor a 0,10 en Macro F1 carece de soporte estadístico con esta cantidad de muestras. La sincronización atacante-ventana descrita en §4.3 (1:N táctica:ventana) introduce dos compromisos de validez interna que deben reportarse: (i) un overhead temporal acumulado por los sleeps de sincronización en cada transición de táctica, típicamente entre 0 y 5 s por transición (acotado por `interval + 1 s` como cap defensivo en `_wait_for_next_window_boundary`), reportado en metadata por corrida; el wall-clock total del atacante es por tanto una cota superior con respecto al wall-clock sin sincronización; (ii) la imposibilidad de evaluar el observer en escenarios donde múltiples tácticas ocurren en la misma ventana de 5 s — un régimen operacionalmente posible pero deliberadamente excluido por la restricción metodológica. Las corridas pre-mayo-2026 reportadas en este trabajo (matriz exploratoria temprana sobre mrrobot, §6.2.2) operaron sin esta sincronización; sus resultados de `strict_accuracy` se interpretan en régimen multi-tactic per window y no son directamente comparables con las corridas del Eje A/B/D bajo régimen 1:N.

Validez externa: los siete escenarios evaluados son aplicaciones web sobre Linux (PHP/WordPress, Java, Drupal, boot2root SSH y PHP eval-stdin). El sistema no se ha evaluado en escenarios de Active Directory, Windows, movimiento lateral o exfiltración. La generalización a otros stacks tecnológicos y a otras tácticas del kill chain MITRE (Persistence, Lateral Movement, Defense Evasion, Collection, Command and Control, Exfiltration, Impact, Resource Development) constituye trabajo futuro.

Validez de conclusión: dos escenarios del experimento (Mr. Robot y DC-1) son CTFs públicos de VulnHub con writeups indexados en motores de búsqueda y disponibles en el corpus de pre-entrenamiento de GPT-4.1 (data cutoff abril 2025). Aunque el prompt del atacante incluye un principio anti-trampa explícito (sección 4.3) que prohíbe invocar credenciales o paths memorizados de writeups conocidos, no se ha cuantificado empíricamente cuánto del éxito del atacante en estos escenarios se debe a razonamiento genuino versus a memorización de soluciones del CTF. El escenario bpent (boot2root construido específicamente para este trabajo, sin walkthrough público) muestra el comportamiento opuesto — el atacante completa solo Reconnaissance y queda atascado en Initial Access — lo que sugiere que parte del desempeño en mrrobot/dc1 puede atribuirse al conocimiento previo del modelo. Cuantificar este efecto mediante variantes "anonimizadas" de los CTFs (con username, paths y credenciales modificados) es trabajo futuro. Esta amenaza se enmarca en la crítica de Happe & Cito (2025) [38] sobre la representatividad de los benchmarks CTF para pentesting real: en su revisión de 19 trabajos de offensive-security LLM advierten que las evaluaciones basadas en CTF públicos arrastran contaminación de corpus que infla artificialmente el desempeño respecto al pentesting en entornos sin walkthrough indexado. La asimetría observada en este trabajo entre mrrobot/dc1 (CTF públicos) y bpent (boot2root construido ad hoc) es consistente con dicho patrón.

6.5 Limitaciones del sistema.

Las limitaciones documentadas en esta sección NO son detalles periféricos: informan directamente sobre la pregunta de investigación y constituyen restricciones estructurales del enfoque LLM-based, no defectos corregibles con prompt engineering o instrumentación menor. Dos limitaciones centrales — el sesgo de frecuencia del LLM ante credenciales no canónicas y la capacidad insuficiente de modelos free-tier para escenarios complejos — emergen como resultados primarios del experimento, no como secundarios. Se documentan a continuación en orden de impacto sobre la pregunta de investigación; las limitaciones operacionales menores se discuten en §6.5.3. Zhang et al. (2025) [18] y Divakaran & Peddinti (2024) [19] documentan limitaciones estructurales análogas en sus revisiones del estado del arte de LLMs en ciberseguridad; este trabajo cuantifica empíricamente la magnitud de dos de ellas. Marco interpretativo — scope conditions y dominio de validez: las limitaciones se reportan bajo el patrón académico establecido por los foros dedicados a fronteras de capacidad de modelos fundacionales (workshop ICBINB en NeurIPS, workshop LASER en USENIX) [34], donde declarar fallos no triviales constituye contribución de pleno derecho. El lenguaje formal de scope conditions (Foschi 1997 [35]) permite distinguir resultados sostenidos por evidencia de regímenes en los que la métrica pierde interpretabilidad. Esta separación es coherente con benchmarks recientes del campo: PentestEval [28] reporta 31 % de pipelines end-to-end exitosos como techo del estado del arte, y Cyber Defense Benchmark [27] reporta 3,8 % de eventos correctamente etiquetados por el modelo frontera Claude Opus 4.6 sobre 86 sub-técnicas. Los dos runs de scope identificados en el Eje B (mrrobot y bpent-GPT4.1, 1/6 tácticas, §6.3.5) caen dentro del régimen documentado por estos benchmarks; se reportan como evidencia del frequency bias del modelo y caracterización del techo operativo, no como medición del observer.

Limitación principal — sesgo de frecuencia del LLM en tareas de enumeración:

El experimento original sobre el escenario `bpent` (`boot2root` construido específicamente para esta evaluación, sin `walkthrough` público) con GPT-4.1 como atacante reveló que el agente completa solamente Reconnaissance (1/6 tácticas) y queda atascado en Initial Access. La causa raíz documentada en §6.3.5 es que el LLM prioriza usernames frecuentes (`admin`, `test`, `user`) sobre el username real del wordlist (`marlinspike`, palabra náutica inusual), incluso cuando este último está literalmente presente en `fsociety.dic` disponible al agente. La corrida C3 del 1 de mayo de 2026 (Sonnet 4.5 atacante y observador) completó 6/6 tácticas en `bpent` — incluyendo el brute force a `marlinspike`, el cracking del hash con `john` y la obtención de `root_flag` — en 102 acciones y 17 m 52 s de wall-clock con 0 replans, lo que demuestra que la limitación es model-dependent y no inherente al enfoque LLM-based. La lectura ajustada del hallazgo es que el sesgo de frecuencia en user enumeration sigue siendo una limitación operacional para modelos como GPT-4.1 y los free-tier evaluados, pero un modelo más capaz puede atravesar el cuello de botella sin instrumentación adicional. Mitigaciones que siguen siendo relevantes para modelos limitados: (a) prompt engineering específico que fuerce enumeración exhaustiva del wordlist antes de probar candidatos comunes; (b) instrumentación con un módulo de selección por frecuencia inversa; (c) reportar usernames poco comunes como hallazgo de Reconnaissance separado. Este patrón es coherente con la literatura empírica sobre memorización en modelos de lenguaje: Carlini et al. (2023) [36] demuestran que la memorización crece log-linealmente con (i) la capacidad del modelo, (ii) el número de duplicados del ejemplo en el corpus de entrenamiento y (iii) la longitud del contexto, lo que justifica la preferencia sistemática del agente por tokens frecuentes (`admin`, `root`, `test`) sobre tokens raros (`marlinspike`). Bender et al. (2021) [37] argumentan que los LLM amplifican distribuciones de frecuencia del corpus sin acceso al sentido subyacente, por lo que el sesgo es estructural y no eliminable por ingeniería de prompt — solo mitigable mediante instrumentación externa o por capacidad superior del modelo subyacente.

Limitación principal — capacidad insuficiente de modelos free-tier en escenarios complejos:

El uso de modelos gratuitos como agente atacante presenta dos restricciones operacionales críticas documentadas en las corridas D1–D8 del 2 de mayo de 2026: (i) capacidad de razonamiento insuficiente para escenarios complejos —gpt-oss-120b en OpenRouter completa dvwa en 4/4 (D7, 34 acciones) pero se atasca en mrrobot (D1, 1/6 tras 38 min), log4shell (D3, 2/3 tras 28 min) y confluence (D4, 1/3 tras 22 min)— y (ii) cuotas de free-tier incompatibles con el tamaño del prompt del sistema: Cerebras Qwen3 235B alcanza el límite tokens-per-day (D5, 2 acciones antes de retries de 60 s), Gemini 2.5 Flash agota la cuota de 20 req/día (D6, 16 acciones), Groq Llama 3.3 70B rechaza la corrida con error 413 porque el prompt del sistema (19 432 tokens) excede el TPM limit (12 000 tokens) del free-tier. Sólo OpenRouter gpt-oss-120b mantiene operación sostenida sobre múltiples corridas, a costa de un wall-clock 2,95× mayor que Sonnet 4.5 en dvwa (598 s vs 202 s). El escenario log4shell completa 2/3 tácticas con 258 tool_calls y 18 replans en 49 minutos de wall-clock en una corrida histórica, lo cual es operacionalmente inviable. Esta limitación significa que el enfoque solo es democrático (accesible sin presupuesto) para escenarios web simples; para escenarios de pentesting realistas se requiere un modelo de pago. La conclusión “el LLM atacante es viable como herramienta autónoma” debe restringirse al régimen modelo-premium o a tareas acotadas de baja complejidad.

Otras limitaciones identificadas:

(a) Lag de detección por ventana temporal corta: cuando el atacante avanza entre tácticas más rápido que el intervalo de polling de 5 segundos, las tácticas intermedias quedan parcialmente observadas. En el escenario Mr. Robot, cinco tácticas ocurren en aproximadamente 30

segundos tras la fase de scanner, lo cual aún reduce la cantidad de ventanas evaluables (e.g., 13 ventanas evaluables para la corrida con observer Sonnet 4.5).

(b) Dominancia de logs de scanner: nikto y gobuster generan hasta 1500 entradas de log en 5 segundos sostenidos, lo que diluye eventos breves de fases avanzadas en el resumen entregado al LLM. Las mitigaciones descritas en §4.4.3 (reestructuración del prompt con bloque "EVENTOS CRÍTICOS DETECTADOS") y §4.4.2 (calibración adaptativa por táctica) mitigan este efecto pero no lo eliminan completamente.

(c) Credential Access invisible al observador cuando el cracking se realiza offline: si el atacante ejecuta `john --wordlist` sobre un hash extraído, esa actividad no genera tráfico HTTP visible al observer y la táctica no puede inferirse. Esta es una limitación arquitectónica del modelo de observables HTTP-only, no del clasificador. La extensión a observables adicionales (eventos de proceso, logs de sistema operativo) es trabajo futuro.

(d) Privilege Escalation se confunde con Discovery cuando los comandos son semánticamente ambiguos sin contexto de intención (ej: `find -perm -4000` puede clasificarse como Discovery o Privilege Escalation según la fase del kill chain). Esta ambigüedad es inherente a la observación HTTP de webshell sin telemetría de proceso, y limita el F1 por táctica en clases tardías.

(e) Conocimiento del LLM sobre CTFs públicos: dos de los seis escenarios (Mr. Robot, DC-1) son CTFs ampliamente documentados con walkthroughs en internet, presumiblemente presentes en el corpus de pre-entrenamiento del modelo. La validación interna y de constructo del experimento (sección 6.4) discute este efecto y propone como trabajo futuro la creación de variantes anonimizadas para cuantificarlo.

CAPÍTULO 7. CONCLUSIONES Y TRABAJO FUTURO

7.1 Logros y contribuciones del trabajo.

El presente trabajo se enmarca como measurement paper, no system paper: la contribución central NO es proponer una arquitectura agéntica nueva, sino medir empíricamente la precisión de inferencia táctica MITRE ATT&CK por un observer LLM mediante telemetría de red y logs del sistema en un entorno adversarial controlado, dentro del régimen documentado por PentestEval [28] y Cyber Defense Benchmark [27] como techo actual del estado del arte. Los componentes individuales del sistema (atacante LLM autónomo, observador desde logs, orquestación con LangGraph, memoria por fingerprint, validators code-based) existen previamente en literatura 2023-2025; la contribución legítima es su composición específica para responder esta pregunta empírica. Las contribuciones principales son cinco. Primero, una arquitectura híbrida que combina pre-clasificación determinística (heurísticas T1-T10 + firmas CVE) con clasificación contextual basada en LLM, siguiendo el patrón Triage → Investigate → Classify → Escalate de Vinay (2025). El experimento de ablation reportado en la sección 6.2.4 cuantifica el aporte de la capa estadística previa al LLM: una mejora absoluta de 0,18 en Macro F1 (de 0,23 a 0,41) y una reducción a un tercio de los falsos positivos en la clase mayoritaria (Reconnaissance).

Segundo, una matriz comparativa de cinco proveedores LLM como observador en condiciones controladas (atacante GPT-4.1 fijo, escenario basic / DVWA, prompts idénticos, reset de memoria entre celdas) que documenta el trade-off latencia/calidad del estado actual del mercado: en el Eje A (basic), Qwen3-235B vía Cerebras free lidera el ranking de observers en μ macro-F1 (0,583 con $n=2$, alta inestabilidad operacional) y DeepSeek-chat es el observer ganador operacional ($\mu=0,479$, latencia 7,94 s, sostenido en todos los escenarios del Eje B). En la fase exploratoria temprana sobre mrrobot (pre-anti-cheating, Tabla 1) Claude Sonnet 4.5 alcanzó Macro F1 = 0,66 con el atacante completando 6/6 tácticas; este resultado histórico no es directamente comparable con la matriz Eje A debido a la diferencia

de régimen (anti-cheating estricto introduce el caveat documentado en §6.2.2). Estos resultados se reportan como tendencias exploratorias por la cantidad limitada de muestras (single-run por celda en el Eje A); su consolidación con múltiples corridas e intervalos de confianza por celda es trabajo futuro.

Tercero, una caracterización empírica del sesgo de frecuencia del LLM en user enumeration: el caso bpent (boot2root con username marlinspike no canónico) atascó al agente con GPT-4.1 en Initial Access (1/6 tácticas, sección 6.5), y al usar Claude Sonnet 4.5 (corrida C3) el escenario se completa 6/6 sin atasco. Este resultado sugiere que el desempeño del agente en tareas de enumeración con candidatos infrecuentes depende fuertemente de la capacidad del modelo subyacente, y que la generalización del enfoque a entornos reales —donde los usernames y credenciales no siguen patrones publicados— exige modelos del nivel de Sonnet 4.5 o superior, o instrumentación específica para forzar exploración exhaustiva con modelos menos capaces. La cuantificación sistemática de este efecto a lo largo del catálogo de modelos disponibles constituye una pregunta abierta del campo. Esta caracterización empírica del sesgo de frecuencia en user enumeration es coherente con la literatura sobre memorización en LLMs (Carlini et al. 2023 [36]; Bender et al. 2021 [37]) y aporta evidencia ofensiva concreta a la crítica de Happe & Cito (2025) [38] sobre la representatividad de benchmarks CTF; el caso bpent (escenario sin walkthrough público) opera como control empírico del efecto de contaminación de corpus en evaluaciones LLM-based de pentesting.

Cuarto, un módulo de métricas académicas (src/evaluation/metrics.py) independiente del orquestador que calcula precision, recall y F1-score micro/macro sobre clasificación multi-label, con metadata embebida (seed, modelo, hash de commit, timestamp) en cada EvaluationReport. Esta infraestructura habilita la verificación post-hoc por terceros y se alinea con ISO/IEC TS 4213:2022 sobre evaluación de clasificadores ML.

Quinto, un repositorio público con código completo, tests unitarios, docker-compose reproducible y datos de evaluación (github.com/Crescendum429/mitre-adversarial-system). El repositorio documenta el ciclo iterativo del proyecto a través de cinco hitos identificables en el historial de commits, con criterios de aceptación verificables por corrida.

7.2 Limitaciones identificadas como aporte.

Más allá de las limitaciones específicas listadas en la sección 6.5, cuatro limitaciones del enfoque LLM-based pueden interpretarse como contribuciones empíricas al campo. (a) La capacidad de un LLM como clasificador táctico depende fuertemente de la ingeniería del prompt: la fase exploratoria temprana sobre mrrobot (pre-anti-cheating, §6.2.2) registra una mejora de aproximadamente 25 puntos absolutos en Macro F1 sobre el mismo modelo entre prompts iniciales y prompts post-mejora (de 0,36 a 0,61 para GPT-4.1; de 0,41 a 0,66 para Sonnet 4.5), magnitud comparable o mayor a las diferencias entre proveedores en la matriz cross-modelo del Eje A. Estas cifras corresponden al régimen comparable interno de la fase exploratoria y no son directamente comparables con los valores del Eje B post anti-cheating; el orden de magnitud del salto, sin embargo, indica que el techo aparente del enfoque está acotado por la ingeniería del prompt antes que por la capacidad del modelo. (b) Los modelos gratuitos (Nemotron 120B, Cerebras Qwen3 235B) son competitivos en F1 absoluto, lo que apoya la viabilidad económica del enfoque para investigadores académicos sin presupuesto comercial. (c) El servicio gratuito de Groq no procesa el prompt actual (~12-14K tokens) devolviendo HTTP 413 — limitación estructural del free tier comercial, no del modelo subyacente. (d) Comparado con baselines clásicos de Machine Learning para la misma tarea (Daniel et al. 2024 sobre 973 reglas Snort), los LLMs presentan métricas inferiores pero aportan capacidades complementarias (zero-shot sobre tácticas no vistas, razonamiento explicable, adaptación contextual) — esto se discute en la sección 6.3.2.

7.3 Trabajo futuro

Las fases siguientes del proyecto abordarán: (a) implementación in-house de baselines clásicos (Random Forest, SVM, XGBoost) sobre las mismas ventanas de observación del Eje A para comparación directa con el observer LLM (complementa la revisión de literatura ya reportada en §6.3.2 que se basa en evaluaciones publicadas por terceros sobre datasets distintos); (b) experimento de ablación con y sin historial de clasificaciones para cuantificar el impacto de la dependencia temporal en la accuracy; (c) implementación de escenarios adicionales que cubran tácticas de Exfiltration, Lateral Movement e Impact; (d) reducción del intervalo de polling a 10-15 segundos para capturar tácticas de corta duración; (e) generación automática de reportes de vulnerabilidades por el Agente Atacante para uso en Red Team real.

REFERENCIAS

- [1] T. Nieponice et al., "ARACNE: An LLM-Based Autonomous Shell Pentesting Agent," arXiv preprint arXiv:2502.18528, Feb. 2025. [Online]. Available: <https://arxiv.org/abs/2502.18528>
- [2] X. Shen, M. Zhang, Y. Li, K. Chen, X. Liu, J. Wang, and T. Zhang, "PentestAgent: Incorporating LLM Agents to Automated Penetration Testing," arXiv preprint arXiv:2411.05185, Nov. 2024. [Online]. Available: <https://arxiv.org/abs/2411.05185>
- [3] L. Huang, J. Wang, X. Chen, Y. Zhang, M. Li, J. Liu, and H. Wang, "RvB: Automating AI System Hardening via Iterative Red-Blue Games," arXiv preprint arXiv:2601.19726, Jan. 2026. [Online]. Available: <https://arxiv.org/abs/2601.19726>
- [4] N. Daniel et al., "Labeling NIDS Rules with MITRE ATT&CK Techniques: Machine Learning vs. Large Language Models," arXiv preprint arXiv:2412.10978, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2412.10978>

- [5] V. Vinay, "The Evolution of Agentic AI in Cybersecurity: From Single LLM Reasoners to Multi-Agent Systems and Autonomous Pipelines," arXiv preprint arXiv:2512.06659, Dec. 2025. [Online]. Available: <https://arxiv.org/abs/2512.06659>
- [6] S. Srinivas, B. Kirk, J. Zendejas, M. Espino, M. Boskovich, A. Bari, K. Dajani, and N. Alzahrani, "AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation," *Journal of Cybersecurity and Privacy*, vol. 5, no. 4, p. 95, Nov. 2025.
- [7] MITRE Corporation, "MITRE ATT&CK Framework," 2025. [Online]. Available: <https://attack.mitre.org/>
- [8] M. Sobieraj and D. Kotyński, "Docker Performance Evaluation across Operating Systems," *Applied Sciences*, vol. 14, no. 15, p. 6672, Jul. 2024. [Online]. Available: <https://www.mdpi.com/2076-3417/14/15/6672>
- [9] A. Mellen and P. Harrington, "MITRE ATT&CK Evaluations Return: More Coverage, More Nuance," *Forrester Research Blog*, Dec. 10, 2025. [Online]. Available: <https://www.forrester.com/blogs/mitre-attck-evaluations-return-more-coverage-more-nuance/>
- [10] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR 2023*, 2023.
- [11] LangChain Inc., "LangGraph Documentation," 2025. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [12] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy," *Technical Report*, The MITRE Corporation, 2018.
- [13] OpenAI, "Introducing GPT-4.1 in the API," *OpenAI Blog*, Apr. 14, 2025. [Online]. Available: <https://openai.com/index/gpt-4-1/>

- [14] Anthropic, "Claude Haiku 4.5 Model Overview," Anthropic, 2025. [Online]. Available: <https://www.anthropic.com/news/claude-haiku-4-5>
- [15] Sapkota et al., "LangChain vs. LangGraph vs. LangSmith: Taxonomies of Agentic AI Toolchains for End-to-End Orchestration," TechRxiv, Sep. 2025. DOI: 10.36227/techrxiv.175695645.52670060. [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.175695645.52670060/v1>
- [16] G. Deng et al., "PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing," in Proc. 33rd USENIX Security Symposium (USENIX Security '24), Aug. 2024. [Online]. Available: <https://arxiv.org/abs/2308.06782>
- [17] J. Huang and Q. Zhu, "PenHeal: A Two-Stage LLM Framework for Automated Pentesting and Optimal Remediation," in Proc. ACM Workshop on Autonomous Cybersecurity (AutonomousCyber '24), Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2407.17788>
- [18] J. Zhang et al., "When LLMs Meet Cybersecurity: A Systematic Literature Review," Cybersecurity, vol. 8, no. 1, Feb. 2025. DOI: 10.1186/s42400-025-00361-w. [Online]. Available: <https://link.springer.com/article/10.1186/s42400-025-00361-w>
- [19] D. M. Divakaran and S. T. Peddinti, "LLMs for Cyber Security: New Opportunities," arXiv preprint arXiv:2404.11338, Apr. 2024. [Online]. Available: <https://arxiv.org/abs/2404.11338>
- [20] hackermentor, "VulnHub Mr. Robot Walkthrough," Medium, s.f. [Online]. Available: <https://medium.com/@hackermentor/vulnhub-mr-robot-walkthrough>

- [21] National Institute of Standards and Technology, "SP 800-94: Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST Special Publication, 2007.
<https://csrc.nist.gov/pubs/sp/800/94/final>
- [22] C. Elkan, "The Foundations of Cost-Sensitive Learning," in Proc. 17th International Joint Conference on Artificial Intelligence (IJCAI), 2001, pp. 973-978.
<https://cseweb.ucsd.edu/~elkan/rescale.pdf>
- [23] J. C. Platt, "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods," in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61-74.
- [24] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network Anomaly Detection: Methods, Systems and Tools," IEEE Communications Surveys & Tutorials, vol. 16, no. 1, pp. 303-336, 2014. DOI: 10.1109/SURV.2013.052213.00046.
- [25] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157-173, 2024. arXiv:2307.03172.
- [26] M. Sokolova and G. Lapalme, "A Systematic Analysis of Performance Measures for Classification Tasks," Information Processing & Management, vol. 45, no. 4, pp. 427-437, 2009. DOI: 10.1016/j.ipm.2009.03.002.
- [27] B. Hou et al., "Cyber Defense Benchmark: Agentic Threat Hunting Evaluation for LLMs in SecOps," arXiv:2604.19533, Feb. 2026.
- [28] Y. Liu et al., "PentestEval: Benchmarking LLM-based Penetration Testing," arXiv:2512.14233, Dec. 2025.
- [29] Y. Liu et al., "AthenaBench: A Dynamic Benchmark for Evaluating LLMs in Cyber Threat Intelligence," arXiv:2511.01144, Nov. 2025.
- [31] Anthropic, "Introducing Claude Sonnet 4.5," Anthropic news release, 2025.

- [34] NeurIPS Workshop on "I Can't Believe It's Not Better" (ICBINB) y USENIX LASER Workshop (Learning from Authoritative Security Experiment Results), foros dedicados a la publicación de resultados negativos y fronteras de capacidad en aprendizaje automático y ciberseguridad respectivamente. [Online]. Disponibles: <https://icbinb.cc/> y <https://www.usenix.org/conference/laser2017>
- [35] M. Foschi, "On Scope Conditions," Small Group Research, vol. 28, no. 4, pp. 535-555, 1997. DOI: 10.1177/1046496497284004.
- [36] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang, "Quantifying Memorization Across Neural Language Models," in Proc. International Conference on Learning Representations (ICLR), 2023. arXiv:2202.07646. [Online]. Disponible: <https://arxiv.org/abs/2202.07646>
- [37] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?," in Proc. ACM Conference on Fairness, Accountability, and Transparency (FAccT '21), 2021, pp. 610-623. DOI: 10.1145/3442188.3445922.
- [38] A. Happe and J. Cito, "Benchmarking Practices in LLM-driven Offensive Security: Testbeds, Metrics, and Experiment Design," arXiv preprint arXiv:2504.10112, Apr. 2025. [Online]. Disponible: <https://arxiv.org/abs/2504.10112>

DECLARACIÓN DE USO DE TECNOLOGÍAS GENERATIVAS Y ASISTIDAS POR IA

Para la elaboración del proyecto integrador titulado "Sistema Adversarial de Simulación de Ataques con Clasificación Automática de Tácticas MITRE ATT&CK Mediante Agentes Autónomos", el autor declara haber utilizado herramientas de IA generativa (Claude, de Anthropic, y modelos de OpenAI) para tareas de asistencia en la implementación del sistema, refinamiento de redacción, corrección gramatical y tipográfica, y diseño de los componentes visuales del frontend (Claude Design). El autor revisó y editó exhaustivamente todo el contenido generado y asume plena responsabilidad por la versión final del documento, el código publicado y los resultados experimentales reportados.

El uso de estas herramientas no exime al autor del cumplimiento de los

principios académicos de originalidad, integridad y rigor científico aplicables al presente trabajo.